
Rapport de projet de fin d'études

CrediSense AI

Plateforme d'aide à la décision pour le crédit à la consommation :
lecture des dossiers par OCR, classification, assistant documentaire (RAG) et
moteur de décision hybride

Réalisé par : Meriem Rehouma

Encadrant académique : [nom]

Encadrant professionnel : [nom]

Organisme d'accueil : [organisme] · contexte applicatif : Attijari Bank

Soutenance : [date]

Remerciements

À rédiger par l'auteur. Cette page est personnelle : remerciements aux encadrants, à l'organisme d'accueil, à la famille. Elle n'est volontairement pas écrite à votre place.

Résumé

L'étude d'une demande de crédit à la consommation oblige aujourd'hui un conseiller à relire cinq ou six pièces, souvent scannées, en arabe et en français, à recalculer à la main le taux d'endettement et à vérifier la cohérence de l'identité. Ce travail est long, répétitif et source d'erreurs. Ce projet réalise **CrediSense AI**, une plateforme qui assiste le conseiller de bout en bout, sans jamais se substituer à lui.

La plateforme se compose d'un portail Angular, d'un backend Spring Boot qui orchestre le traitement et d'un service d'intelligence artificielle FastAPI. Les pièces sont lues par un **OCR** (PaddleOCR, arabe et français), classées par une **cascade à trois niveaux** (règles, embeddings, modèle de langage), structurées par extraction JSON adaptée au type de document, puis exploitées par un **assistant documentaire (RAG)** à recherche hybride. Le cœur du système est un **moteur de décision hybride** : le modèle de langage lit et rédige, mais tous les chiffres et toutes les décisions réglementaires sont recalculés par des règles déterministes, versionnées et testées. Chaque décision est relue par un humain avant d'être envoyée, tracée dans un journal d'audit, et suivie par un tableau de bord avec export Excel.

L'évaluation porte sur un jeu de documents fictifs. Sur ce jeu, l'OCR retrouve la totalité des champs attendus, y compris sur les scans, mais reste lent sur processeur (de l'ordre de 40 secondes par page). Les mesures détaillées de chaque étape, la méthode pour les reproduire et leurs limites sont présentées au chapitre 4. L'application est déployée sur Microsoft Azure avec Terraform, Docker Compose et un reverse proxy HTTPS, et mise à jour par GitHub Actions.

Mots-clés : crédit à la consommation, OCR, classification de documents, RAG, recherche hybride, moteur de règles, modèle de langage, aide à la décision, traçabilité, déploiement cloud.

Abstract

Assessing a consumer-credit application forces a loan officer to read five or six documents, often scanned, in Arabic and French, to recompute the debt ratio by hand and to cross-check the applicant's identity. **CrediSense AI** assists the officer from end to end without replacing them. The platform combines an Angular portal, a Spring Boot orchestration backend and a FastAPI AI service. Documents go through OCR (PaddleOCR, Arabic and French), a three-level classification cascade (rules, embeddings, language model), type-aware JSON extraction and a hybrid-retrieval question-answering assistant (RAG). The core is a **hybrid decision engine**: the language model reads and writes, but every figure and regulatory verdict is recomputed by deterministic, versioned, tested rules. Each decision is reviewed by a human before sending, recorded in an append-only audit log and summarised in a dashboard with Excel export. Evaluation on a fictitious document set shows complete field recall in OCR, with a processing time of about 40 seconds per scanned page on CPU. The system is deployed on Microsoft Azure with Terraform, Docker Compose and GitHub Actions.

Keywords: consumer credit, OCR, document classification, retrieval-augmented generation, hybrid search, rule engine, large language model, decision support, auditability, cloud deployment.

Sommaire

Introduction générale

Chapitre 1 — Contexte et cadrage du projet

- Contexte

- Problématique

- Objectifs et périmètre

- Acteurs et besoins

- Exigences

- Méthodologie de travail

Chapitre 2 — Conception et architecture

- Vue d'ensemble

- Choix technologiques

- Architecture du backend et modèle de données

- Architecture du service d'intelligence artificielle

- Chaîne de traitement d'un dossier

- Principes de conception

- Sécurité

Chapitre 3 — Réalisation

- OCR

- Classification des documents

- Extraction de données
- Assistant documentaire (RAG)
- Moteur de décision hybride
- Contrôle humain, notification et audit
- Statistiques et export
- Interface utilisateur

Chapitre 4 — Tests et évaluation des performances

- Stratégie de tests
- Résultats des tests automatisés
- Protocole de mesure
- Indicateurs de performance par étape
- Analyse, limites et menaces sur la validité
- Synthèse à présenter en soutenance

Chapitre 5 — Déploiement

- Environnement cible
- Infrastructure en tant que code
- Conteneurisation
- Intégration et déploiement continu
- Configuration et secrets
- Mise en ligne d'une version
- Mesures de déploiement

Conclusion générale et perspectives

Annexes

- Glossaire
- Points d'entrée de l'API
- Variables de configuration
- Speech de présentation de l'architecture
- Commandes de reproduction des mesures

Liste des abréviations

Sigle	Signification
API	Application Programming Interface
BCT	Banque Centrale de Tunisie
BM25	Best Matching 25 : fonction de classement par mots exacts
CER / F1	Character Error Rate / moyenne harmonique de la précision et du rappel
CI/CD	Intégration continue / déploiement continu
CIN	Carte d'identité nationale
DTI	Debt-to-Income : taux d'endettement (charges mensuelles ÷ revenu net)
FAISS	Facebook AI Similarity Search : bibliothèque de recherche de vecteurs
JWT	JSON Web Token
KPI	Key Performance Indicator : indicateur clé de performance
LLM	Large Language Model : grand modèle de langage
MDH	Moteur de décision hybride (règles déterministes + modèle de langage)
MRR	Mean Reciprocal Rank : rang moyen inverse de la première bonne réponse
OCR	Optical Character Recognition : reconnaissance optique de caractères
RAG	Retrieval-Augmented Generation : génération appuyée sur une recherche documentaire
RRF	Reciprocal Rank Fusion : fusion de classements
SSR	Server-Side Rendering : rendu côté serveur

Introduction générale

Un établissement bancaire qui accorde un crédit à la consommation doit répondre à une question en apparence simple : *ce client peut-il rembourser ?* Pour y répondre, un conseiller réunit une carte d'identité, une fiche de paie, un relevé bancaire, une attestation de travail et un justificatif de domicile. Il lit chaque pièce, vérifie que l'identité est la même partout, repère les crédits déjà en cours, calcule la mensualité et le taux d'endettement, puis confronte le résultat aux seuils réglementaires. Les pièces sont souvent des scans, en arabe et en français ; le calcul est répété à chaque dossier ; l'erreur d'un chiffre peut conduire à accorder un crédit qui ne sera pas remboursé ou à refuser un client solvable.

L'intelligence artificielle générative semble offrir un raccourci : confier le dossier à un modèle de langage et lui demander une décision. Ce raccourci est dangereux dans un contexte bancaire. Un modèle de langage peut inventer un chiffre, ne donne pas deux fois la même réponse et ne peut pas justifier précisément une décision. Le projet part donc d'un constat inverse : **le modèle de langage est excellent pour lire et rédiger, mauvais pour calculer et décider**. L'architecture est construite autour de cette séparation.

Ce rapport présente **CrediSense AI**, une plateforme qui assiste le conseiller sans jamais le remplacer. Il est organisé en cinq chapitres. Le **chapitre 1** pose le contexte, la problématique, les objectifs et les exigences. Le **chapitre 2** décrit la conception et l'architecture. Le **chapitre 3** détaille la réalisation, étape par étape : OCR, classification, extraction, assistant documentaire, moteur de décision, contrôle humain et audit, statistiques. Le **chapitre 4** présente la stratégie de tests et, surtout, les **indicateurs de performance mesurés** pour chaque étape, avec la méthode pour les reproduire et leurs limites. Le **chapitre 5** décrit le déploiement sur Microsoft Azure. Une conclusion dresse le bilan et les perspectives.

Convention de lecture. Tout chiffre de performance de ce rapport est soit *mesuré* (avec la date, le jeu de données et la commande qui le produit), soit signalé comme *à mesurer*. Aucun chiffre n'est estimé ou inventé. Les mesures portent sur un **jeu de documents fictifs** généré pour le projet : elles prouvent le bon fonctionnement de la chaîne, pas sa précision sur de vrais dossiers bancaires.

Chapitre 1 — Contexte et cadrage du projet

Ce chapitre situe le projet : le contexte bancaire, le problème à résoudre, les objectifs fixés, les acteurs concernés et les exigences qui en découlent.

1.1 Contexte

Le crédit à la consommation est un produit de masse : les montants sont modestes, les dossiers nombreux, les délais de réponse attendus par les clients courts. En Tunisie, les pièces justificatives sont fournies en arabe, en français ou dans les deux langues, sous forme de PDF natifs, de scans ou de photos. L'étude d'un dossier repose sur des règles prudentielles de la Banque Centrale de Tunisie (BCT) : plafond d'endettement, durée maximale, plafond du montant en fonction du salaire, âge maximal en fin de crédit.

Le travail du conseiller se décompose en quatre tâches : **lire** les pièces, **contrôler** leur cohérence, **calculer** la capacité de remboursement, **décider** et **répondre** au client. Les trois premières sont mécaniques et se prêtent à l'automatisation ; la quatrième engage la responsabilité de la banque.

1.2 Problématique

La question centrale est : **comment automatiser la lecture et le calcul d'un dossier de crédit avec des techniques d'intelligence artificielle, tout en gardant une décision fiable, explicable et contrôlée par un humain ?** Elle se décline en cinq difficultés.

#	Difficulté	Conséquence si elle n'est pas traitée
1	Les pièces sont hétérogènes (natif, scan, photo, arabe, français)	Texte illisible ou incomplet, donc données fausses
2	Le client choisit lui-même où il dépose chaque pièce	Un relevé déposé comme fiche de paie fausse le revenu et les dettes
3	Un modèle de langage invente, varie et n'explique pas ses calculs	Décision non reproductible, indéfendable devant un audit
4	Une donnée absente peut être confondue avec une valeur nulle	Un dossier incomplet sort « éligible » (dette inconnue comptée comme 0)
5	La banque doit rendre des comptes sur chaque décision	Impossible de savoir qui a décidé quoi, quand, avec quelles règles

Tableau 1.1 — Les cinq difficultés à résoudre

1.3 Objectifs et périmètre

Objectifs

1. **Lire** automatiquement les pièces d'un dossier, natives ou scannées, en arabe et en français.
2. **Vérifier** la nature de chaque pièce et la cohérence de l'identité.
3. **Calculer** mensualité, taux d'endettement, capacité d'emprunt et simulations, de façon reproductible.
4. **Proposer** une décision (éligible, conditionnel, refus, à compléter) expliquée critère par critère.
5. **Aider** le conseiller à interroger le dossier en langage naturel (assistant documentaire).
6. **Contrôler et tracer** : validation humaine avant l'envoi, journal d'audit, tableau de bord.
7. **Déployer** l'ensemble sur un cloud public, de façon reproductible.

Périmètre

Dans le périmètre	Hors périmètre
Crédit à la consommation ; cinq types de pièces (CIN, fiche de paie, relevé bancaire, attestation d'emploi, justificatif de domicile) ; trois rôles ; réponse au client par e-mail avec PDF ; déploiement sur une VM Azure	Crédit immobilier ; connexion à la centrale des risques de la BCT ; signature électronique ; grille de score propriétaire de la banque ; validation sur de vrais dossiers bancaires

Honnêteté sur les seuils : le moteur est un prototype. Les seuils du moteur (30 % et 35 % d'endettement, 5 fois le salaire, 84 mois, 6 mois d'ancienneté, 70 ans) et les poids de la grille de score sont des **valeurs par défaut indicatives** : ils n'ont pas été confrontés à la politique de crédit de la banque ni aux circulaires de la BCT. Ils sont regroupés dans **un seul fichier de paramètres** où chaque seuil porte sa **source**, la personne qui l'a **validé** et la **date de validation**. Tant que ces champs sont vides, le statut est « PROTOTYPE » et l'application l'affiche à l'agent (écran et PDF).

1.4 Acteurs et besoins

Acteur	Besoin	Fonctions offertes
Client sans compte, via le portail public	Déposer une demande et ses pièces, recevoir une réponse claire	Création de la demande, dépôt des pièces par type, réponse par e-mail avec rapport PDF
Agent conseiller bancaire, rôle AGENT	Étudier vite et sûrement un dossier, rester maître de la décision	Vérification des documents, analyse, résultat expliqué, assistant documentaire, validation avant envoi, journal du dossier
Administrateur rôle ADMIN	Piloter l'activité et gérer les agents	Gestion des agents, tableau de bord statistique, export Excel

Tableau 1.2 — Acteurs, besoins et fonctions

1.5 Exigences

Exigences fonctionnelles

Réf.	Exigence	Chapitre
EF1	Lire un PDF natif, un scan ou une photo, en arabe et en français	3.1
EF2	Classer chaque pièce et signaler un désaccord avec le type déclaré	3.2
EF3	Extraire les données utiles, avec un schéma adapté au type de pièce	3.3
EF4	Vérifier que le CIN lu est celui du client, sur chaque pièce	3.3, 3.6
EF5	Répondre à des questions sur le dossier avec les sources	3.4
EF6	Calculer mensualité, endettement, capacité, simulations ; rendre une décision expliquée	3.5
EF7	Proposer un montant ou une durée ajustés pour un dossier conditionnel	3.5
EF8	Faire valider une réponse par un agent avant l'envoi ; pouvoir l'annuler	3.6
EF9	Journaliser chaque décision et chaque action ; fournir statistiques et export	3.6, 3.7

Exigences non fonctionnelles

Réf.	Qualité	Exigence
ENF1	Fiabilité des chiffres	Un chiffre financier est reproductible : même entrée, même sortie
ENF2	Honnêteté	Une donnée inconnue n'est jamais traitée comme nulle ; aucune valeur inventée
ENF3	Traçabilité	Journal en ajout seul, version des règles consignée avec chaque décision
ENF4	Disponibilité	Une panne du modèle de langage ou de la messagerie ne perd aucun dossier
ENF5	Performance	Étapes déterministes en millisecondes ; temps de l'OCR mesuré et documenté
ENF6	Confidentialité	CIN, RIB et IBAN masqués avant tout envoi à un modèle externe ; rôles ADMIN et AGENT
ENF7	Maintenabilité	Chaque module est testé ; la chaîne de déploiement est reproductible

1.6 Méthodologie de travail

Le projet a été conduit de façon itérative : chaque lot de fonctionnalités a été développé avec ses tests, mesuré, puis livré dans un commit. Les décisions de conception ont été prises sur des **mesures** et non sur des intuitions. Exemple : les embeddings seuls classaient une attestation de travail comme une carte d'identité (similarité 0,407 contre 0,403 pour la bonne réponse) ; cette mesure a conduit à ajouter un niveau de règles en amont. Chaque étape de la chaîne est évaluée indépendamment avant d'être branchée dans le pipeline.

Chapitre 2 — Conception et architecture

Ce chapitre présente l'architecture du système, les choix technologiques et les principes qui guident la conception. Il se termine par le déroulement complet du traitement d'un dossier.

2.1 Vue d'ensemble

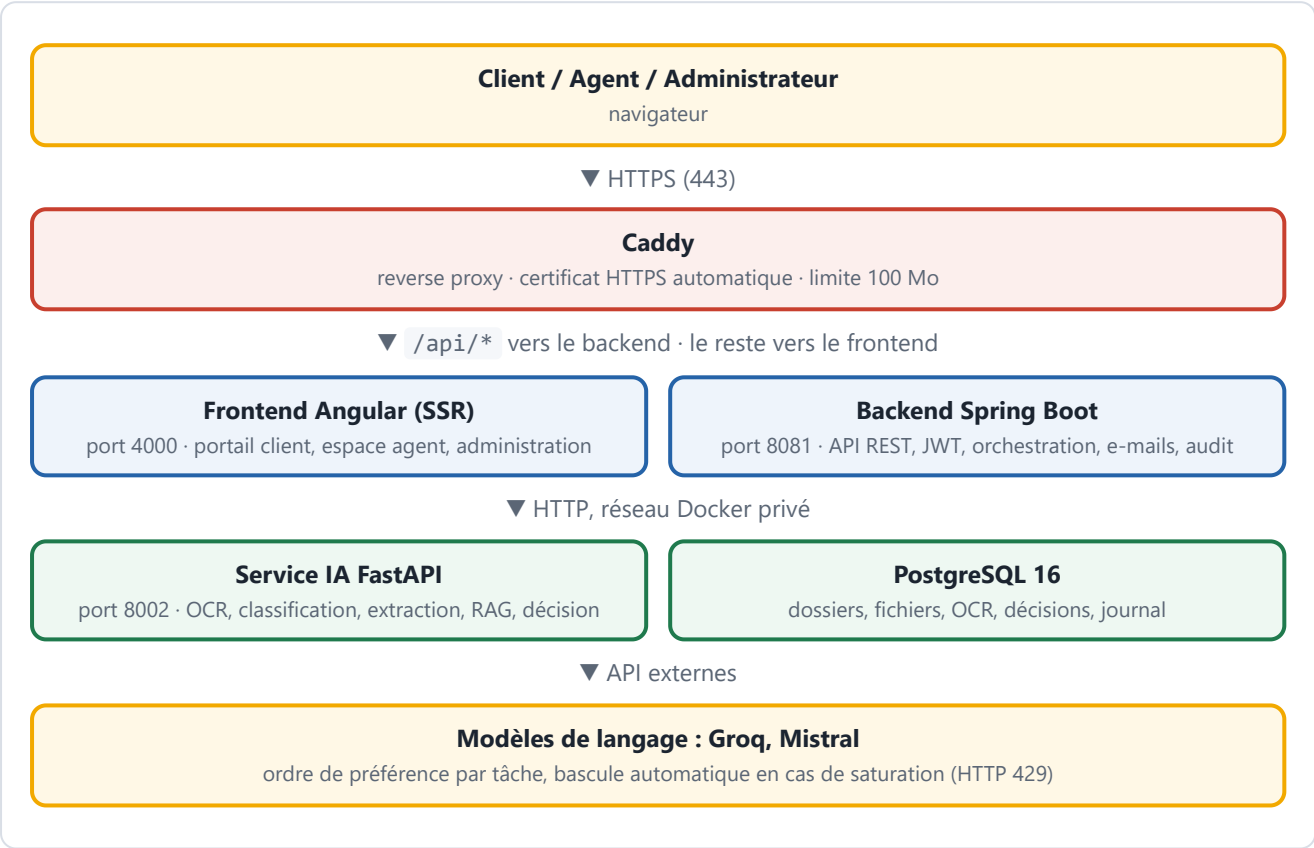


Figure 2.1 — Architecture de déploiement : six conteneurs sur une machine virtuelle

Rôle de chaque brique

Brique	Rôle	Technologies
Frontend	Portail client (demande, dépôt des pièces), espace agent (pipeline, résultat, chatbot), administration (agents, statistiques)	Angular, composants autonomes, SSR
Backend	Authentification, dossiers, fichiers, orchestration du pipeline , cohérence du CIN, notification, journal d'audit, statistiques, PDF et Excel	Spring Boot, JPA/Hibernate, JWT, iText, Apache POI
Service IA	OCR, classification, extraction, assistant documentaire, moteur de décision	FastAPI, PaddleOCR, PyMuPDF, MiniLM, FAISS, Pydantic
Base de données	Source de vérité : tout est reconstituible depuis elle (hors index mémoire du chatbot)	PostgreSQL 16

Proxy	Termine le HTTPS, route les requêtes	Caddy 2
-------	--------------------------------------	---------

Tableau 2.1 — Les briques du système

2.2 Choix technologiques

Choix	Justification	Alternative écartée
Deux langages : Java et Python	Le monde métier (sécurité, transactions, base) est solide en Java ; l'écosystème IA (OCR, embeddings) vit en Python. Les deux se parlent par HTTP simple et se déploient séparément.	Tout en Python : moins de maturité côté sécurité et transactions
PaddleOCR	Modèles arabe et latin, exécution locale (les pièces d'identité ne quittent pas le serveur pour la lecture), précision constatée sur nos scans	OCR en ligne : données personnelles envoyées à un tiers
MiniLM multilingue	Embeddings 384 dimensions, français et arabe, rapide sur processeur	Modèles plus gros : latence et mémoire
FAISS + BM25	Le sens (FAISS) et les mots exacts (BM25) se complètent ; les montants et numéros sont mal pesés par les seuls embeddings	Recherche dense seule
Règles déterministes pour les chiffres	Reproductibilité, explicabilité, testabilité (section 2.6)	Décision laissée au modèle de langage
Plusieurs fournisseurs de LLM	Bascule automatique si l'un est saturé : le service reste disponible	Fournisseur unique
PostgreSQL	Transactions, types JSON, maturité	Base documentaire : peu adaptée aux jointures du modèle
Docker Compose sur une VM	Adapté à la charge et au budget d'un abonnement étudiant ; reproductible	Kubernetes : surdimensionné ici

Tableau 2.2 — Choix technologiques et alternatives

2.3 Architecture du backend et modèle de données

Le backend suit une architecture en couches classique : **contrôleurs** (API REST) → **services** (logique métier) → **dépôts** (accès aux données JPA) → **entités**. L'authentification est assurée par des jetons JWT, avec deux rôles (ADMIN et AGENT) ; le portail client public passe par des routes dédiées.

Entité	Contenu
Client	Identité du demandeur (nom, prénom, CIN, e-mail)
Dossier	Demande de crédit : montant, durée, contrat, statut (en attente, en cours, approuvé, refusé)
Fichier	Pièce déposée : type déclaré, type détecté, confiance, méthode, conflit, trace de la classification (JSON)

OcrResult	Texte brut et nettoyé, statut (SUCCESS ou FAILED), raison de l'échec
JsonExtraction	Données structurées extraites d'un fichier, score de confiance
AgentAnalysis	Synthèse de l'analyse : score, revenus, endettement, décision
DecisionFinale	Décision, justification, résultat complet en JSON , état de l'envoi (statut, mode, acteur, heure prévue), date de la dernière décision
AuditEvent	Journal en ajout seul : type, acteur, décision, score, version des règles, détail
Agent	Conseiller ou administrateur (rôle, mot de passe chiffré)

Tableau 2.3 — Principales entités

Choix notable : le **résultat complet de l'analyse est conservé en JSON** avec la décision. Il permet de régénérer à tout moment le rapport PDF et les statistiques sans relancer l'analyse, et de relire exactement ce que l'agent a vu.

2.4 Architecture du service d'intelligence artificielle

Module	Responsabilité
ocr_service.py	Détection du format par octets, lecture native (PyMuPDF) ou OCR (PaddleOCR), fusion arabe et latin, cache par empreinte SHA-256
classification_regles.py nlp_service.py llm_classifier_service.py document_classifier_service.py	Les trois niveaux de la cascade et leur orchestration, avec trace niveau par niveau
controle_type.py	Comparaison type déclaré / type détecté, seuil de fiabilité par méthode
groq_service.py	Extraction JSON : schéma et consignes par type de document, validation Pydantic, score de confiance
dossier_texte.py dettes_releve.py	Budget de texte réparti entre les documents ; détecteur de dettes par règles dans le relevé
decoupage_rag.py recherche_hybride.py chatbot_service.py	Découpage structurel et sémantique, BM25 et fusion RRF, assistant documentaire avec mémoire
agent_service.py	Moteur de décision hybride : lecture par LLM, règles déterministes, garde-fous, propositions d'ajustement
llm_client.py	Routeur de fournisseurs de modèles de langage avec bascule automatique

Tableau 2.4 — Modules du service IA

2.5 Chaîne de traitement d'un dossier

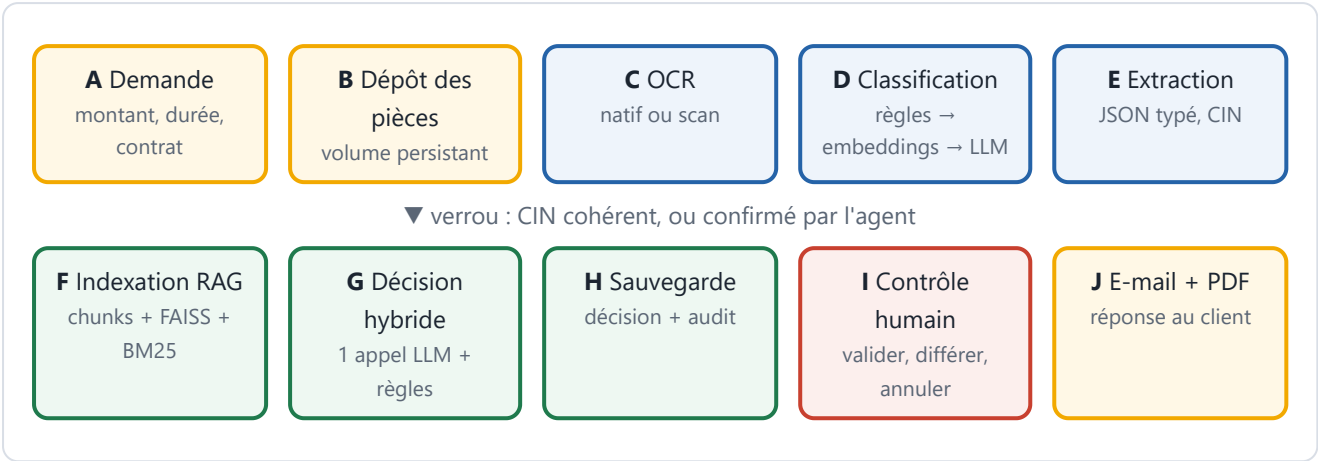


Figure 2.2 — Chaîne de traitement, du dépôt à la réponse

- A Demande.** Le portail envoie le montant, la durée et le type de contrat ; un dossier est créé en base (statut `EN_ATTENTE`).
- B Dépôt.** Chaque pièce est écrite dans un volume persistant, avec un nom de fichier et un CIN assainis, et enregistrée avec le type choisi par le client.
- C-E « Vérifier les documents ».** Pour chaque pièce non encore traitée : OCR, enregistrement du texte, classification (le type détecté est comparé au type déclaré), extraction JSON avec le **type retenu**, puis comparaison, chiffre par chiffre, du CIN lu au CIN du client. Une extraction vide ou en échec n'est pas enregistrée : un document ne peut donc pas être « validé » à tort.
- Verrou.** « Analyser » n'est actif que si les étapes précédentes sont valides. Si le CIN est incohérent ou si un type est contredit, l'agent peut poursuivre après une confirmation explicite ; le serveur applique la même règle (réponse 409 sans confirmation).
- F-G « Analyser ».** Indexation du dossier pour l'assistant, assemblage du texte (un en-tête par document, la demande du client en tête), vérification d'identité, puis appel du moteur de décision.
- H-J Décision et réponse.** Le résultat complet est enregistré, le statut du dossier mis à jour et l'événement ajouté au journal. Selon la configuration, la réponse part immédiatement, après un délai annulable, ou attend la validation d'un agent (refus par défaut).

2.6 Principes de conception

Principe	Application
Le LLM lit, les règles décident	Le modèle extrait et rédige. Mensualité, endettement, capacité, simulations, propositions d'ajustement et verdicts réglementaires sont recalculés en Python, sans modèle.
Ne jamais inventer l'inconnu	Taux d'intérêt obligatoire ; dette inconnue $\neq$ dette nulle ; pas de calcul sans les données, mais une décision « À COMPLÉTER » qui liste ce qui manque.

Humain dans la boucle	Un désaccord de type ou de CIN bloque jusqu'à confirmation ; un refus attend la validation ; une proposition de montant est indicative.
Traçabilité	Journal en ajout seul ; version des règles (2026-10-c) consignée avec chaque décision ; trace de la cascade de classification conservée.
Dégradation maîtrisée	Chaque panne (LLM, messagerie, classification) est absorbée : le traitement continue avec ce qui est disponible, l'erreur est enregistrée et visible.
Moindre exposition	L'e-mail part du résultat <i>enregistré</i> et non de ce que le navigateur envoie ; CIN, RIB et IBAN sont masqués avant tout appel à un modèle externe.

Tableau 2.5 — Principes de conception

2.7 Sécurité

- **Authentification et rôles** : jetons JWT, rôles ADMIN et AGENT ; routes d'administration réservées à ADMIN.
- **Secrets** : clés d'API, SMTP, JWT et mot de passe de base fournis par fichiers d'environnement sur la VM, jamais dans le code.
- **Données personnelles** : masquage des CIN, RIB et IBAN avant les appels externes ; le PDF n'est pas stocké (régénéré à la demande) ; le journal ne garde que l'identifiant du dossier.
- **Transport** : HTTPS automatique (Let's Encrypt via Caddy).

Points de sécurité non traités à ce jour. Les routes de réponse aux propositions sont protégées (rôle CLIENT, dossiers du client seulement) et les routes des agents sont réservées aux agents. En revanche, plusieurs routes restent ouvertes sans authentification (/api/fichiers/** , /api/chatbot/** , la plupart des routes /api/clients/** , dont l'historique accessible avec une simple adresse e-mail) ; la route de lecture /api/fichiers/view/** sert un fichier à partir d'un chemin (risque de traversée de répertoire) ; des pièces d'identité de test sont suivies par git ; des clés d'API ont été exposées pendant le développement et doivent être renouvelées. Ces points sont à corriger avant tout usage réel (voir la conclusion).

Chapitre 3 — Réalisation

Ce chapitre décrit comment chaque étape de la chaîne a été réalisée : le problème posé, la solution retenue, son fonctionnement et ses limites. Les résultats chiffrés sont regroupés au chapitre 4.

3.1 OCR : lire les pièces natives et scannées

Problème

Une pièce peut être un PDF « natif » (texte sélectionnable), un scan sans texte, ou une photo. Elle peut mêler arabe et français. L'extension du fichier ne dit rien de son contenu : un fichier renommé en `.pdf` peut être un document Word.

Solution

1. **Détection du format par les premiers octets** (PDF, JPEG, PNG, TIFF, BMP, WebP). Un format non supporté (Word, HEIC) est refusé avec un message clair affiché à l'agent.
2. **Détection natif ou scanné** : un PDF dont le texte est exploitable est lu directement avec PyMuPDF (quelques dizaines de millisecondes). Sinon chaque page est rendue en image.
3. **OCR PaddleOCR** : détection des zones de texte, puis lecture de chaque zone par le modèle **arabe** et par le modèle **latin**, et fusion zone par zone selon la confiance. Les zones sont regroupées par ligne et le texte est normalisé (Unicode). Les images sont réduites au-delà de 4 000 pixels et la taille d'un fichier est limitée à 25 Mo.
4. **Cache** par empreinte SHA-256 du fichier : relire un fichier identique est instantané.
5. **Persistence** : le texte, le statut (`SUCCESS` ou `FAILED`) et la raison d'un échec sont enregistrés par fichier, une seule ligne par fichier.

Limites

La fusion de deux modèles double le coût de lecture : c'est la principale cause de la lenteur mesurée au chapitre 4. L'ordre de lecture d'un texte mêlant arabe et latin peut différer de l'ordre visuel, ce qui n'empêche pas l'extraction (qui travaille sur le sens) mais interdit de mesurer l'OCR par distance d'édition caractère à caractère (voir 4.3).

3.2 Classification des documents

Problème

Le client choisit l'emplacement de chaque pièce. Un relevé bancaire déposé dans « fiche de paie » serait traité comme une fiche de paie : revenu et dettes seraient faux, et le calcul de décision avec.

Solution : une cascade à trois niveaux

Niveau	Fonctionnement	Coût
--------	----------------	------

0. Règles	Mots-clés par type, en trois forces : titres (« attestation de travail », « bulletin de paie »...), forts, faibles. Le poids d'un titre dépend de sa position : plein en haut du texte, réduit plus bas (une attestation qui cite « la carte d'identité nationale » n'est pas une CIN). Texte normalisé, français et arabe. Les règles ne répondent que si le verdict est net (score ≥ 5 et écart ≥ 3) et ne répondent jamais « AUTRE ».	instantané, gratuit, reproductible
1. Embeddings	Similarité (MiniLM multilingue) avec des phrases de référence ; verdict retenu au-dessus de 0,55.	quelques ms, local
2. LLM	Seulement en zone grise (similarité entre 0,20 et 0,55) ; en cas de panne, le verdict des embeddings est conservé.	un appel externe, rare

Tableau 3.1 — Les trois niveaux de la classification

Contrôle déclaré / détecté. Un conflit n'est signalé que si le verdict est **fiable**, avec un seuil par méthode car les échelles de confiance ne sont pas comparables (règles 0,80 ; embeddings 0,55 ; LLM 0,70). Un verdict incertain, « AUTRE » ou un type non déclaré ne bloque rien. En cas de conflit fiable, le **type détecté pilote l'extraction** ; le type déclaré n'est pas écrasé, et l'analyse reste verrouillée jusqu'à la confirmation de l'agent.

Traçabilité. Chaque verdict est accompagné de la **trace de la cascade** (niveau consulté ou non, a tranché ou non, pourquoi). Elle est stockée avec le fichier et affichée à l'agent dans le popup « Détail de la classification » ; une ligne lisible apparaît aussi sur chaque document (« Détecté : Relevé bancaire · règles par mots-clés · confiance 90 % »).

3.3 Extraction de données

Le texte de chaque pièce est transformé en JSON par un modèle de langage, avec deux précautions. D'abord, le **schéma demandé dépend du type retenu** : une CIN ne demande que le nom, le prénom et le numéro (demander quinze champs à une CIN pousse le modèle à en inventer) ; une fiche de paie demande le « net à payer » et pas le brut ; un relevé demande les échéances de crédit et les incidents. Ensuite, la réponse est **validée par Pydantic** et accompagnée d'un score de confiance calculé selon les champs attendus pour ce type.

Cohérence du CIN. Le backend compare, en chiffres uniquement, le CIN extrait de chaque pièce au CIN du client : cohérent, incohérent ou sans objet. **Vérification d'identité** : le nom et le prénom du client sont comparés à ceux lus dans les pièces par ensemble de mots, sans tenir compte de l'ordre, des accents ni de la casse (une fiche client enregistrée avec nom et prénom inversés ne déclenche plus de fausse alerte).

Budget de texte. Le dossier est envoyé au modèle avec un budget de 12 000 caractères réparti équitablement entre les documents ; un document trop long garde son début et sa fin, aucun n'est supprimé, et tout raccourcissement est signalé.

Détecteur de dettes. Les crédits en cours sont repérés *par règles* dans le relevé (échéances de prêt regroupées par date, salaire et solde exclus, plus forte somme mensuelle sur les deux derniers mois) et comparés à la valeur lue par le modèle : on retient la plus élevée, avec un avertissement en cas d'écart. Un « 0 » sans relevé lisible n'est pas une preuve : la dette reste *inconnue*.

3.4 Assistant documentaire (RAG)

L'agent peut poser des questions au dossier en langage naturel (« quel est le net à payer ? », « y a-t-il un crédit en cours ? »). On ne donne pas tout le dossier au modèle à chaque question : on *cherche* d'abord les passages utiles, puis on les lui fournit avec la question.

Indexation et découpage

Type de texte	Découpage
Relevé bancaire	Un chunk par opération , avec répétition de l'en-tête des colonnes et déduction du sens crédit/débit à partir du solde (jamais inventé si le solde ne colle pas)
Document court	Un seul chunk, jamais coupé
Texte libre	Coupure aux changements de sujet (semantic chunking : similarité des embeddings avant et après chaque frontière)
Tous	450 caractères au plus, ce que le modèle d'embedding lit réellement

Tableau 3.2 — Découpage structurel et sémantique

Les chunks sont transformés en vecteurs (MiniLM, 384 dimensions) rangés dans un index **FAISS** (recherche exacte par cosinus), doublé d'un index lexical **BM25** écrit en Python. Un index existe par dossier, en mémoire.

Recherche hybride

- **Sens** (FAISS) : « salaire » retrouve « net à payer », en français comme en arabe.
- **Mots exacts** (BM25) : montants, noms et numéros, que les embeddings pèsent mal.
- **Fusion RRF** : un chunk bien classé par l'une ou l'autre recherche remonte.
- **Couverture** : au moins deux extraits par document, puis complément jusqu'à douze.

Mémoire de conversation et garde-fous

Les six derniers échanges sont transmis au modèle, et une question « relance » (« Et depuis quand ? ») est cherchée avec la ou les questions précédentes. L'historique vient du navigateur : il n'est pas cru (seuls les rôles « user » et « assistant » sont gardés, les identifiants sensibles masqués, les messages tronqués). CIN, RIB et IBAN sont masqués avant l'envoi au modèle et dans la réponse ; le prompt impose de ne répondre que d'après le dossier.

3.5 Moteur de décision hybride (MDH)

Nom de cette partie. L'ancien nom « agent IA de décision » laissait croire à un agent autonome qui prend des décisions. Ce n'est pas le cas, et c'est volontaire. Le nom retenu est « **moteur de décision hybride** » : *hybride* parce qu'il combine un modèle de langage (qui lit et rédige) et des règles déterministes (qui calculent et tranchent) ; *moteur* parce qu'il applique des règles versionnées et testables, sans autonomie d'action. Alternative possible : « module d'aide à la décision de crédit ».

Partage des rôles

Le modèle de langage...	Les règles déterministes...
lit le dossier et en extrait revenu, contrat, ancienneté, dettes ; rédige le résumé, les points forts, les risques et le plan ; propose une décision	recalculent mensualité, taux d'endettement, capacité d'emprunt et simulations ; calculent le score ; vérifient chaque critère ; corrigent la décision du modèle ; listent les données manquantes ; proposent des ajustements

Formules

- **Mensualité** (annuité constante) : $M = C \times i / (1 - (1 + i)^{-n})$, avec $i = \text{taux annuel} \div 12$.
- **Taux d'endettement** : $\text{DTI} = (M + \text{dettes mensuelles}) \div \text{revenu net} \times 100$.
- **Mensualité maximale** : $30 \% \times \text{revenu} - \text{dettes}$; le **montant finançable** en découle par inversion de la formule d'annuité, plafonné à 5 fois le salaire.
- **Simulations** : durées de 12, 24, 36, 48, 60 et 84 mois et durée demandée ; mensualité, DTI, coût total et statut pour chacune.

Contrôles réglementaires

Contrôle	Seuil	Bloquant	Si la donnée manque
Taux d'intérêt appliqué	taux configuré, entre 0 et 50 %	oui	aucun calcul, « à renseigner »
Taux d'endettement	< 30 % OK · 30–35 % attention · > 35 % KO	oui	« inconnu », rien d'inventé
Dettes existantes	relevé bancaire lisible	oui	« à confirmer »
Plafond du montant	$\leq 5 \times \text{salaire net}$	oui	à vérifier
Durée	≤ 84 mois	oui	à vérifier
Ancienneté dans l'emploi	≥ 6 mois	oui	à vérifier (attestation)
Type de contrat	CDI / fonctionnaire favorable	non	à vérifier
Incidents de paiement	0	si > 0	à vérifier (centrale des risques)
Âge en fin de crédit	≤ 70 ans	oui	contrôle non affiché

Tableau 3.3 — Contrôles du moteur de décision

Décisions possibles et garde-fous

Décision	Sens	Statut du dossier
ELIGIBLE	Tout est connu, aucun critère bloquant en échec	APPROUVE
CONDITIONNEL	Acceptable avec garanties, ou un critère bloquant en échec	EN_COURS
REFUS	Refus fondé sur un critère connu	REFUSE

A_COMPLETER	Des informations indispensables manquent : aucune décision rendue, liste des manques, score « provisoire » plafonné à 59	EN_COURS
INDETERMINE	Le modèle de langage a échoué	EN_COURS

Tableau 3.4 — Les décisions

Deux garde-fous s'appliquent après le modèle : un critère bloquant en échec ramène « ELIGIBLE » à « CONDITIONNEL » (score plafonné à 59), et des données indispensables manquantes transforment toute décision favorable en « À COMPLÉTER ». Un cache dont la clé contient le texte, la **version des règles**, le taux et la taille maximale évite de resservir une ancienne analyse après un changement.

Paramètres du moteur et statut « prototype »

Les seuils d'acceptation ne sont plus écrits dans le code mais dans un fichier JSON (`parametres_regles.json`). Pour chaque seuil : sa valeur, son unité, sa **source**, le **validateur** et la **date de validation**. Le fichier est vérifié au chargement (seuils cohérents, poids totalisant 100, fractions entre 0 et 1) ; s'il est absent ou invalide, le service ne s'arrête pas : il utilise les valeurs internes et le signale dans le résultat.

- **Statut** : « PROTOTYPE » tant qu'un seuil n'a pas de validateur et de date ; « VALIDE » seulement quand le fichier est marqué ainsi et que chaque seuil est signé et daté. Le statut, la liste des seuils non validés et une **empreinte** des valeurs sont ajoutés à chaque résultat.
- **Cache** : l'empreinte fait partie de la clé de cache, donc modifier un seuil ne ressort jamais une ancienne analyse.
- **Instruction du modèle de langage** : elle reprend les seuils du fichier, il n'existe plus de seconde copie à tenir à jour.

Score par grille à points

Problème mesuré. Le score donné par le modèle de langage changeait d'un appel à l'autre pour le même dossier (78 puis 85) et ne s'expliquait pas. **Solution.** Le score est une somme de points par critère, calculée par le code (`score_grille.py`) ; le score proposé par le modèle n'est conservé qu'à titre indicatif.

Critère	Points max.	Calcul (valeurs par défaut du prototype)
Taux d'endettement	40	plein jusqu'à 20 % ; 60 % des points à 30 % ; 20 % à 35 % ; 0 point à 40 % (interpolation linéaire entre ces repères)
Stabilité du contrat	20	CDI et fonctionnaire : 100 % ; retraité : 70 % ; CDD : 50 % ; indépendant : 40 %
Ancienneté dans l'emploi	15	paliers : moins de 6 mois : 0 ; 6 mois : 33 % ; 12 mois : 50 % ; 24 mois : 80 % ; 60 mois : 100 %
Incidents de paiement	15	aucun : 100 % ; un : 33 % ; plus : 0
Montant demandé / salaire	10	plein jusqu'à 2 fois le salaire ; 30 % au plafond de 5 fois ; 0 au-delà

Tableau 3.3 bis — Grille de score (poids et paliers modifiables dans le fichier de paramètres)

- **Reproductible et explicable** : même dossier, même note ; l'agent voit « 95 = 40 + 20 + 15 + 15 + 5 » dans une carte « Détail du score » et dans le PDF agent.
- **Jamais de donnée devinée** : un critère inconnu (incidents non vérifiés, date d'embauche absente...) n'est ni compté comme zéro ni inventé ; il est **écarté**, le score est rapporté aux critères connus et marqué « provisoire ».
- **Cohérence avec la décision** : un critère bloquant en échec plafonne le score à 59 ; une décision « ELIGIBLE » avec un score définitif inférieur à 60 est ramenée à « CONDITIONNEL » (avec un risque expliquant pourquoi).

Prototype. Les poids et les paliers sont un choix de conception : une banque aurait sa propre grille. Ils sont affichés comme « à valider avec la banque » dans l'application, le PDF agent et ce rapport.

Propositions d'ajustement pour un dossier conditionnel

Pour un dossier CONDITIONNEL, le moteur calcule jusqu'à trois variantes, chacune **revérifiée** avec les mêmes règles : **montant réduit** à durée égale (arrondi aux 100 DT inférieurs) ; **même montant sur une durée allongée** (la plus courte, par pas de 6 mois, qui passe sous 30 %) ; **durée maximale et montant ajusté** lorsque l'allongement seul ne suffit pas. L'âge en fin de crédit borne la durée, le plafond de 5 fois le salaire borne le montant. Aucune offre n'est faite sur une donnée inconnue ; si l'endettement est déjà correct, le moteur explique que le caractère conditionnel vient d'autres critères et les liste. Les offres sont **indicatives** pour l'agent. Une fois la réponse envoyée au client (après validation d'un agent si la configuration l'exige), elles figurent dans l'e-mail et dans le PDF client, et le client peut les accepter ou les refuser depuis son espace (section 3.6).

Limites. Les poids de la grille et les seuils ne sont pas validés par la banque (prototype). Le texte rédigé par le modèle de langage (résumé, analyse détaillée) peut encore citer ses propres chiffres, qui peuvent différer de ceux du moteur : l'analyse détaillée est donc repliée et marquée « indicative » dans l'écran de l'agent, et l'e-mail et le PDF d'un dossier conditionnel s'appuient sur le résumé et sur les chiffres du moteur.

3.6 Contrôle humain, notification et audit

Réponse au client

Dès que la décision est enregistrée, le serveur génère le **rapport PDF version client** (iText) et l'envoie par e-mail. Le PDF est régénéré à la demande à partir du résultat enregistré (jamais périmé, aucune donnée personnelle de plus au repos). Il en existe deux versions : **client** (décision, explication, informations à fournir, chiffres de la demande, sans contrôles internes ni risques) et **agent** (rapport complet). Un chiffre non calculé s'écrit « Non calculé », jamais 0.

Contrôle humain avant l'envoi

Cas	Comportement
-----	--------------

Décision soumise à validation (par défaut : refus)	État « en attente de validation » ; l'agent choisit « Valider et envoyer » ou « Ne pas envoyer » ; l'envoi direct est masqué
Autre décision, délai à 0	Envoi immédiat
Autre décision, délai de N minutes	État « programmé » : envoi à l'échéance par une tâche planifiée (toutes les 30 s), annulable avant
Nouvelle analyse, même décision	Aucun changement (pas de remise en attente, pas de renvoi, pas de résurrection d'une réponse annulée)
Nouvelle analyse, décision différente	Nouveau cycle pour la nouvelle décision

Tableau 3.5 — Comportement de l'envoi

Garde-fou. « Valider » envoie au serveur la décision que l'agent a sous les yeux ; si une nouvelle analyse l'a changée, le serveur répond 409 et n'envoie rien : le client ne reçoit jamais autre chose que ce qui a été relu.

Réponse du client aux propositions

Pour un dossier CONDITIONNEL qui a des propositions, l'**e-mail** détaille les chiffres du moteur (montant, durée, mensualité, endettement), les conditions à remplir et chaque option, avec un bouton vers l'espace client. Dans « Mes demandes », une alerte « Une proposition attend votre réponse » ouvre une fenêtre où le client choisit une option, puis **accepte** (après une étape de confirmation) ou **refuse** l'ensemble.

Règle	Application
Quand elles sont visibles	Seulement après l' envoi de la réponse (un agent qui a mis la réponse en validation garde la main), pour une décision CONDITIONNEL dont le moteur a calculé au moins une offre
Qui peut répondre	Le client connecté, sur ses propres demandes (route réservée au rôle CLIENT ; une demande qui n'est pas la sienne répond « introuvable »)
Une seule réponse	La seconde tentative reçoit une erreur 409 ; l'offre choisie est conservée telle que le client l'a vue
Nouvelle analyse	Si la décision ou l'offre choisie a changé, la réponse est effacée et le journal le consigne
Côté agent	Un bandeau « Le client a accepté une proposition » (ou « refusé ») avec l'offre ; le dossier n'est pas modifié automatiquement
Journal d'audit	Offre acceptée, propositions refusées, réponse réinitialisée

Sécurité. Le serveur reconnaît désormais le jeton du client (rôle CLIENT). En contrepartie, les routes des agents (/api/dossiers/** et toute route non publique) sont réservées aux rôles ADMIN et AGENT : un client connecté ne peut atteindre aucune route d'agent. Neuf tests chargent la vraie configuration de sécurité pour le vérifier.

Journal d'audit

La table `audit_events` est en **ajout seul** : entité immuable, aucun endpoint de modification ni de suppression, dossier référencé par son seul identifiant, écriture qui ne fait jamais échouer l'opération décrite. Elle consigne l'analyse (décision, score, version des règles, fournisseur d'IA, taux appliqué, informations manquantes), la confirmation d'une incohérence, la mise en attente, la validation, l'annulation, l'envoi ou son échec (destinataire, mode, PDF joint), le téléchargement du rapport et le changement de statut. L'acteur est l'agent connecté, ou « SYSTÈME » pour une tâche automatique. Une frise « Journal du dossier » l'affiche à l'agent.

3.7 Statistiques et export

Un onglet « Statistiques » de l'espace administrateur, alimenté par `GET /api/admin/statistiques` (rôle ADMIN, 30 jours par défaut, 800 jours au plus), présente le taux d'acceptation (éligibles ÷ décisions définitives, absent et non 0 % s'il n'y en a aucune), la répartition des décisions, les délais de traitement (dépôt → décision) et d'envoi, les motifs de refus, l'évolution par jour ou par semaine et l'état des envois. L'**export Excel** (Apache POI) contient quatre feuilles (Synthèse, Décisions avec filtre et en-tête figé, Motifs, Évolution), de vraies dates et de vrais nombres, et jamais le CIN.

Les graphiques sont en SVG sans bibliothèque, avec marques fines, grille discrète, une infobulle par colonne (souris et clavier) et une vue en tableau de chaque graphique ; aucune information ne repose sur la couleur seule.

3.8 Interface utilisateur

Écran	Contenu
Portail client	Demande de crédit, dépôt des pièces par type, confirmation
Pipeline de l'agent	Quatre étapes (vérification du CIN, validation des documents, analyse, score) réellement verrouillées ; message « déposé comme X, le contenu ressemble à Y » ; popup de classification
Résultat	Décision, score (« provisoire » si incomplet), chiffres clés (« Non calculé » plutôt que 0), contrôles, capacité, propositions d'ajustement, simulations, points forts et de vigilance, bandeau d'envoi, journal du dossier, export PDF
Assistant	Conversation avec mémoire et sources
Administration	Agents et tableau de bord statistique

Tableau 3.6 — Écrans principaux

Chapitre 4 — Tests et évaluation des performances

Ce chapitre répond à deux questions : **le système fait-il ce qu'il doit faire** (tests de fonctionnement) et **à quel niveau de performance** (indicateurs mesurés, étape par étape). Les deux sont distincts : un test passe ou échoue ; un indicateur se mesure et se compare à un objectif.

4.1 Stratégie de tests

Ce qu'il faut tester à chaque étape

Pour chaque étape de la chaîne, on teste le **fonctionnement** (cas normaux, cas limites, pannes) et on mesure la **performance** (exactitude, vitesse). Le tableau suivant résume ce qui a été retenu, et sert de modèle pour toute étape ajoutée.

Étape	Ce qu'on teste (fonctionnement)	Ce qu'on mesure (performance)	Jeu de données
OCR	Format détecté par octets, formats refusés, fichier absent, PDF natif et scan, cache, arabe et français	Champs attendus retrouvés ; F1 des mots ; temps par page, natif et scan	5 pièces natives + 5 scannées du dossier fictif
Classification	Chaque niveau seul ; cascade (court-circuit, zone grise, LLM en panne) ; contrôle déclaré/détecté ; trace	Exactitude, F1 macro, couverture et précision des règles, répartition des niveaux, latence par niveau, taux d'appel au LLM	26 documents étiquetés (français, arabe, natifs, scans, types hors dossier)
Extraction	Schéma par type, identité toujours demandée, cache distinct par type, réponse vide non enregistrée	Valeurs exactes retrouvées (revenu, dettes) sur le dossier fictif	Dossier fictif, valeurs attendues connues
RAG	Découpage (une opération par chunk, rien perdu), BM25, RRF, couverture des documents, mémoire, masquage	Rappel@k et MRR (dense, lexical, hybride) ; exactitude des réponses ; latence de recherche et de réponse	22 questions avec réponse attendue sur les 5 documents
Décision	Taux obligatoire, dette inconnue, garde-fous, critères, cache versionné, offres valides	Écart du calcul à une référence indépendante ; offres invalides ; stabilité du score ; latence	20 000 profils simulés ; dossier fictif avec le vrai modèle de langage
Notification, audit	Validation, annulation, délai, doublons, journal immuable, 409	(fonctionnel) — état exact à chaque transition	Tests unitaires et de câblage

Interface	Verrous du pipeline, popups, bandeaux, graphiques, géométrie	(fonctionnel)	Tests de composants
Déploiement	Build des trois images, santé des conteneurs	Taille des images, mémoire, latence HTTP, temps de démarrage	La machine virtuelle réelle

Tableau 4.1 — Que tester et que mesurer, étape par étape

Pyramide de tests et outils

Niveau	Outil	Contenu
Unitaires Python	<code>unittest</code> , modèles simulés	Modules du service IA ; embeddings et LLM remplacés par des simulations déterministes : aucun téléchargement, aucun appel réseau
Unitaires et d'intégration Java	JUnit 5, Mockito	Services, contrôleurs, câblage des services entre eux ; PDF et Excel relus après génération
Composants Angular	Vitest	Rendu et comportement de chaque écran
Évaluation de performance	<code>benchmarks/mesurer_kpi.py</code>	Vrais modèles (OCR, MiniLM, FAISS), vrai modèle de langage, jeu de test fictif
Mesures de déploiement	<code>deploy/mesurer_vm.sh</code>	Machine réelle, lecture seule

4.2 Résultats des tests automatisés

231 / 231 tests Python	296 / 297 tests Java (1 hors périmètre)	227 / 227 tests Angular de nos écrans (235 / 241 au total)	OK construction du frontend
----------------------------------	---	--	---------------------------------------

Couche	Tests	Détail par module
Service IA (Python)	231	score_grille (39) · parametres_regles (30) · agent_regles (21) · propositions_ajustement (17) · classification_regles (24) · decoupage_rag (19) · recherche_hybride (18) · chatbot_memoire (17) · dettes_releve (12) · controle_type (10) · extraction_par_type (10) · dossier_texte (9) · chatbot_indexation (5)
Backend (Java)	296	PropositionClientService (24) · EmailConditionnel (12) · SecurityConfigAccesTest (9) · JwtAuthFilter (8) · ClientPropositionController (12) · FichierServiceImpl (28) · NotificationValidation (29) · StatistiquesService (26) · DossierControllerRapport (23) · RapportPdfService (20) · AuditService (17) · NotificationDecisionService (15) · StatistiquesExcel (14) · AdminStatistiquesController (11) · ResultatEmailService (10) · IdentiteClient (9) · ChatbotServiceMemoire (9) · NotificationWiring (5) · DossierControllerEmail (3) · DossierControllerFichiers (3)

Frontend (Angular)	227	historique (47) · statistiques (48) · credit-result (70) · pipeline-analyse (23) · upload-section (17) · export-button (8) · chat-assistant (7) · modèle de résultat (7)
-----------------------	-----	--

Tableau 4.2 — Tests automatisés (résultats au 8 octobre 2026)

Échecs connus, non liés à nos changements. **DemoApplicationTests** (Java) exige une vraie base PostgreSQL : il échoue hors de l'environnement complet. Six tests Angular générés par défaut (`app` , `document` , `register` , `verify-email` , deux `reset-password`) échouent : aucun de ces fichiers n'a été modifié ; il faut soit les écrire pour de bon, soit les retirer.

Ce que les tests automatisés ne prouvent pas. Les modèles d'embeddings et de langage sont simulés dans les tests unitaires : ils vérifient la logique du code, pas la qualité des modèles. C'est le rôle des mesures de la section 4.4, faites avec les vrais modèles. Les derniers écrans (bandeau de validation, journal, panneau des propositions) sont testés en structure, mais n'ont pas été relus dans un navigateur.

4.3 Protocole de mesure

Matériel et logiciels

Toutes les mesures des sections 4.4.1 à 4.4.4 ont été faites **sur le poste de développement**, sur processeur, sans GPU (voir les caractéristiques exactes en tête de chaque fichier de résultats). Les temps absolus dépendent de cette machine ; les **rapports** entre étapes et les **taux** (exactitude, rappel) en dépendent beaucoup moins. Les temps sur la machine Azure sont à relever par le script de l'annexe E (section 4.4.6).

Jeux de données

Jeu	Contenu
Dossier fictif (<code>test-data/dossier-12015060</code>)	5 pièces en deux versions : natives (texte sélectionnable) et scannées (images avec léger bruit, 7 pages), toutes avec la mention « DOCUMENT FICTIF ». Les valeurs attendues (CIN, noms, salaire net 2 100 DT, dette de 250 DT par mois, etc.) sont connues et documentées.
Banc de classification (<code>Python/tests/documents</code>)	16 documents étiquetés par leur nom, dont des types absents du dossier de test (assurance-vie, contrat de travail, déclaration fiscale, titre de séjour, menu de restaurant), en français et en arabe, natifs et scannés.

Tableau 4.3 — Jeux de données d'évaluation

Biais à connaître. Ces documents ont été générés par nous : leurs titres ressemblent à nos mots-clés, donc les résultats sont **optimistes** par rapport à de vrais documents. Les mesures valident la chaîne ; elles ne remplacent pas une évaluation sur un corpus annoté de la banque.

Définitions des indicateurs

Indicateur	Définition
Champs retrouvés	Part des valeurs attendues (numéro de CIN, noms, montants, dates...) présentes dans le texte lu, quelle que soit la mise en forme (espaces, casse, accents)
F1 des mots	Moyenne harmonique du rappel et de la précision d'un « sac de mots » du texte lu par OCR, comparé au texte natif du même document. Choisi à la place du taux d'erreur par caractère : l'ordre de lecture d'un texte mêlant arabe et latin diffère entre le texte natif et l'OCR, ce qui gonflerait artificiellement la distance d'édition.
Exactitude	Documents correctement classés ÷ documents
F1 macro	Moyenne, sur les types présents, des F1 par type : n'est pas dominée par les types fréquents
Rappel@k	Part des questions dont la réponse figure dans les k premiers extraits
MRR	Moyenne de 1 ÷ (rang du premier bon extrait) : 1,0 = toujours en première position
Médiane, P95	Temps tel que 50 % (respectivement 95 %) des appels sont plus rapides

Tableau 4.4 — Indicateurs

Reproductibilité

Les mesures sont produites par un script unique, `Python/benchmarks/mesurer_kpi.py`, qui écrit un fichier JSON par étape dans `Python/benchmarks/resultats/` (date, machine et résultats détaillés). Les commandes sont à l'annexe E. Les mesures qui appellent un modèle de langage (zone grise de la classification, réponses de l'assistant, décision de bout en bout) utilisent le fournisseur configuré, avec les **seuls documents fictifs** ; l'option `--sans-11m` les désactive.

4.4 Indicateurs de performance par étape

Les mesures ont été faites le 8 octobre 2026 sur un poste Windows 11, processeur Intel de 11^e génération à 8 cœurs, sans GPU. Les fichiers de résultats détaillés sont dans `Python/benchmarks/resultats/`.

4.4.1 OCR

Question : l'OCR retrouve-t-il les informations utiles, et en combien de temps ? Protocole : les 5 pièces du dossier fictif en version native et en version scannée (7 pages scannées) ; 18 valeurs attendues (CIN, noms, date de naissance, montants, employeur, adresse...).

18 / 18

champs retrouvés sur les scans (100 %)

0,997

F1 moyen des mots, scan comparé au texte natif

44,5 s

par page scannée (médiane par document 44,7 s)

24 ms

par document natif (médiane)

Document	Pages	Natif	Scan	Scan par page	Champs (scan)	F1 mots
----------	-------	-------	------	---------------	---------------	---------

CIN	1	36 ms	32,0 s	32,0 s	5 / 5	0,990
Fiche de paie	1	24 ms	44,7 s	44,7 s	3 / 3	1,000
Relevé bancaire	3	20 ms	145,1 s	48,4 s	3 / 3	1,000
Attestation d'emploi	1	20 ms	48,2 s	48,2 s	4 / 4	0,993
Justificatif de domicile	1	39 ms	41,8 s	41,8 s	3 / 3	1,000
Total	7	138 ms	311,8 s	44,5 s	18 / 18	0,997

Tableau 4.5 — OCR : exactitude et temps (chargement des modèles : 9,7 s, une seule fois)

Lecture. La **qualité** est excellente sur ce jeu : toutes les valeurs attendues sont retrouvées, y compris sur les scans. La **vitesse** est le point faible : un scan prend environ 1 800 fois plus de temps qu'un PDF natif, car chaque zone est lue par deux modèles (arabe et latin) avant fusion. Un dossier de cinq pièces scannées demande ainsi plus de 5 minutes d'OCR, contre 138 ms pour les mêmes pièces natives.

Limites. Les scans sont *générés* (images propres avec léger bruit), pas de vraies photos de documents : sur des photos de téléphone (flou, ombres, inclinaison), la qualité sera inférieure. Le temps dépend du processeur ; sur la machine Azure (2 vCPU) il est à relever (section 4.4.6).

4.4.2 Classification des documents

Question : la cascade identifie-t-elle correctement la nature des pièces, et quel niveau fait le travail ?
Protocole : 26 documents étiquetés (16 du banc d'essai dont 5 types absents du dossier de test, 10 du dossier fictif), texte obtenu par l'OCR réel, classés par chaque niveau seul puis par la cascade.

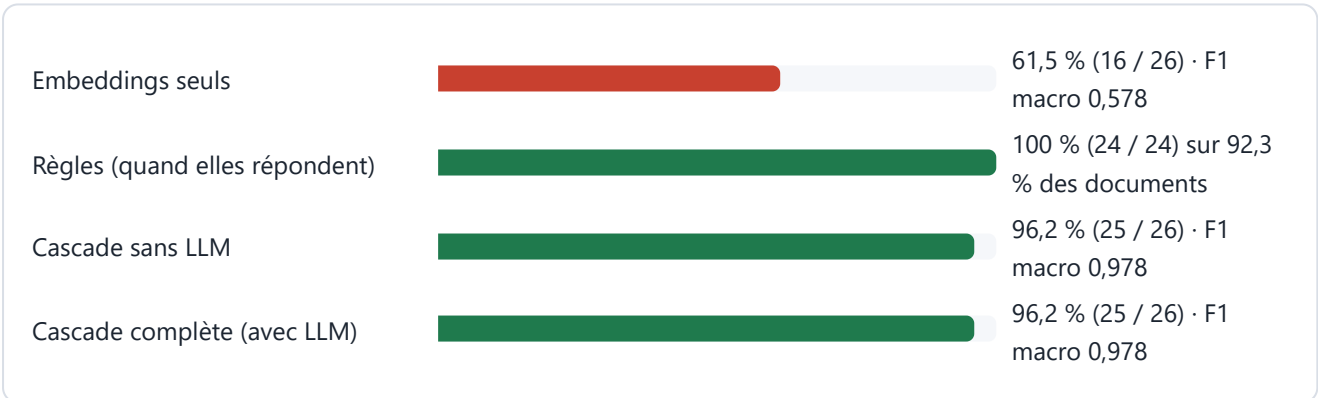


Figure 4.1 — Exactitude de la classification (26 documents)

Indicateur	Valeur
Documents tranchés par les règles	24 sur 26 (92,3 %), sans aucune erreur
Documents tranchés par les embeddings	1 sur 26
Appels au modèle de langage	1 sur 26 (3,8 %) : le menu de restaurant, correctement rangé « AUTRE »
Latence des règles	médiane 2 ms, P95 16,5 ms

Latence des embeddings seuls	médiane 81 ms, P95 112 ms
Latence de la cascade complète (sans appel externe)	moyenne 7,8 ms, P95 63 ms
Latence d'un appel au modèle de langage	932 ms (un seul appel)

Tableau 4.6 — Classification : couverture et vitesse

Ce que les mesures ont justifié. Les embeddings seuls ne classent correctement que 61,5 % des documents : 6 des 10 erreurs sont des pièces classées « CIN » (trois relevés bancaires, deux attestations d'emploi et un titre de séjour citent un numéro d'identité). Le niveau de règles, ajouté après cette constatation, porte l'exactitude à 96,2 % et rend la classification quasi instantanée et gratuite pour 92 % des documents.

Une erreur silencieuse subsiste. L'attestation d'emploi en arabe

(ATTESTATION_EMPLOI__natif_ar) est classée « justificatif de domicile » : les règles s'abstiennent, les embeddings répondent avec une similarité au-dessus du seuil de 0,55 et le modèle de langage n'est donc pas consulté. C'est le cas le plus dangereux : un verdict faux avec une confiance affichée « suffisante ». Piste : relever le seuil, ou consulter le modèle de langage chaque fois que les règles s'abstiennent. Le contrôle déclaré/détecté limite le risque : le client ayant déposé ce document comme attestation d'emploi, le désaccord avec le type détecté (fiable, car au-dessus de 0,55) serait signalé à l'agent, qui devrait confirmer pour poursuivre.

Biais. Les règles ont été écrites en regardant une partie de ces documents : 100 % de précision quand elles répondent est **optimiste**. Seule une évaluation sur de vrais documents de la banque donnera la précision réelle.

4.4.3 Assistant documentaire (RAG)

Question : l'assistant retrouve-t-il les bons passages, et répond-il juste ? Protocole : les 5 documents natifs du dossier fictif sont indexés ; 22 questions dont la réponse attendue est connue (12 sur des montants, numéros et dates, 10 en tournure libre, par exemple « Combien gagne le client par mois ? »). Une question est réussie si la valeur attendue figure dans les k premiers extraits.

Mode de recherche	Rappel@1	Rappel@3	Rappel@5	MRR
Sens seul (embeddings + FAISS)	59,1 %	77,3 %	86,4 %	0,699
Mots exacts seuls (BM25)	59,1 %	77,3 %	77,3 %	0,667
Hybride (fusion RRF)	68,2 %	86,4 %	95,5 %	0,792

Tableau 4.7 — Recherche, toutes questions (22)

Type de question	Sens seul	Mots exacts	Hybride
------------------	-----------	-------------	---------

Montants, numéros, dates (12) — rappel@5 / MRR	83,3 % / 0,644	91,7 % / 0,833	100 % / 0,854
Tournures libres (10) — rappel@5 / MRR	90,0 % / 0,767	60,0 % / 0,467	90,0 % / 0,717

Tableau 4.8 — Recherche selon le type de question

Lecture. La recherche hybride apporte +9 points de rappel@5 (95,5 % contre 86,4 %) et +0,09 de MRR par rapport au sens seul. Elle est surtout utile pour les **valeurs exactes** (100 % de rappel@5) : la question « Quel est le numéro de CIN ? » ramène le bon extrait au rang 2 en hybride, au rang 36 avec le sens seul. Pour les tournures libres, la recherche hybride **n'améliore pas** le sens seul (même rappel@5, MRR légèrement inférieur : 0,717 contre 0,767) : les mots exacts n'aident pas quand la question ne contient pas les mots du document.

Indicateur	Valeur
Rappel dans les 12 extraits réellement envoyés au modèle	95,5 % (21 questions sur 22)
Documents → extraits	5 documents → 45 extraits ; 172 caractères en moyenne, 422 au plus (limite 450) ; 32 extraits viennent du relevé (un par opération)
Indexation d'un dossier (5 documents)	4,2 s (découpage sémantique compris)
Latence de recherche	médiane 20 ms, P95 32 ms
Réponses générées (12 questions)	9 obtenues ; 7 justes sur 9 (77,8 %) ; 3 sans réponse (modèle de langage saturé, HTTP 429)
Temps d'une réponse complète	médiane 5,1 s, P95 14,6 s (dominé par le modèle de langage externe)

Tableau 4.9 — Assistant : extraits, latence et réponses

Sur les 2 réponses non justes. L'une est **voulue** : à la question sur le numéro de CIN, l'assistant ne le donne pas, car les numéros d'identité sont masqués avant et après le modèle (protection des données). L'autre est une vraie erreur (montant de la facture d'électricité). Les 3 questions sans réponse proviennent de la saturation du fournisseur externe, pas de la recherche. L'échantillon (12 réponses) est petit : le 77,8 % est indicatif.

4.4.4 Moteur de décision hybride

Question : les chiffres sont-ils exacts, les propositions valides, la décision stable, et à quelle vitesse ?
Protocole en trois volets.

a) Exactitude du calcul

300 crédits tirés au hasard (montants de 1 000 à 60 000 DT, 6 à 84 mois, taux de 5 à 15 %) : la mensualité du moteur est comparée à une **référence indépendante**, trouvée par dichotomie sur un tableau d'amortissement (la formule n'y intervient pas). Écart maximal : **0,000000 DT** (inférieur à 10^{-6}).

b) Cohérence interne et vitesse sur 20 000 dossiers simulés

0 offre invalide sur 12 912 offres générées	38 µs moteur de règles complet (médiane 36,6 µs ; P95 42,8 µs)	15 µs calcul des propositions (médiane)	77,2 % des dossiers trop endettés reçoivent au moins une offre
--	--	--	--

Parmi les 20 000 dossiers, 10 061 dépassaient 30 % d'endettement ; 7 767 d'entre eux (77,2 %) ont reçu au moins une offre. Chaque offre a été revérifiée à la main : endettement sous 30 %, plafond de 5 fois le salaire, durée au plus 84 mois, âge de 70 ans au plus en fin de crédit, montant inférieur ou égal à la demande. Les 22,8 % restants n'ont **aucune offre** parce qu'aucun montant d'au moins 500 DT ne respecte ces limites (le moteur prévoit ces cas : dettes en cours trop lourdes, âge, plafond du montant ; la répartition exacte entre ces causes n'a pas été mesurée) : il le dit au lieu d'inventer une offre.

c) De bout en bout, avec le vrai modèle de langage, sur le dossier fictif (5 essais)

Indicateur	Valeur
Analyses abouties	2 sur 5 : les 3 autres ont reçu HTTP 429 des deux fournisseurs (quota d'appels dépassé) et sont ressorties « indéterminées » avec un message clair, sans panne ni perte
Revenu lu : 2 100 DT (exact)	2 sur 2
Dettes détectées : 250 DT (exact)	2 sur 2
Taux d'endettement : 22,77 % (référence indépendante : 22,77 %)	2 sur 2
Décision	« ELIGIBLE » dans les 2 cas
Score du modèle de langage (<i>avant la grille à points</i>)	78 et 85 (écart-type 3,5) : les chiffres sont identiques d'un essai à l'autre, le score non . Corrigé ensuite par la grille (voir d)).
Durée d'une analyse aboutie	10,3 s et 19,7 s (un appel externe)

Tableau 4.10 — Décision de bout en bout sur le dossier fictif

À lire avec prudence. Deux analyses abouties seulement : c'est trop peu pour une statistique, mais suffisant pour montrer l'**essentiel** : les valeurs financières sont exactes et stables parce qu'elles viennent des règles, alors que le score varie parce qu'il vient du modèle. Les échecs 429 sont dus aux limites des offres gratuites des fournisseurs, atteintes par la rafale de mesures de la même session. Ils illustrent un risque d'exploitation réel : **la disponibilité du service dépend de fournisseurs externes** (voir 4.5).

d) Score par grille à points

La grille est évaluée sur 2 000 répétitions du même dossier et sur 5 000 dossiers simulés (mesure du 8 octobre 2026, fichier `kpi_decision.json`).



Avant la grille, le score du modèle de langage valait 78 puis 85 pour le même dossier (écart-type 3,5, mesure du tableau 4.10). Avec la grille, le même dossier donne toujours la même note, et chaque point est expliqué par un critère. **Limite** : cette mesure prouve la reproductibilité, pas la pertinence des poids, qui restent un choix de prototype à valider avec la banque. Le nombre de dossiers simulés n'est pas représentatif d'une clientèle réelle.

4.4.5 Chaîne complète : où va le temps

Pour un dossier de cinq pièces **scannées**, la somme des temps mesurés donne l'ordre de grandeur suivant (poste de développement) :



Figure 4.2 — Répartition du temps d'un dossier scanné (somme des mesures isolées, non un chronométrage de bout en bout)

Conclusion de cette répartition : l'OCR représente l'essentiel du temps de traitement (plus de 90 %). Les étapes déterministes sont négligeables. Toute optimisation doit donc viser l'OCR en premier (4.5).

4.4.6 Déploiement

Les mesures sur la machine virtuelle (taille des images, mémoire, latence HTTP, temps de démarrage) sont décrites en 5.7 et se relèvent avec `deploy/mesurer_vm.sh`. [À reporter dans le tableau 5.2 avant la soutenance.](#)

4.5 Analyse, limites et menaces sur la validité

Ce que les résultats établissent

- **Les chiffres financiers sont fiables** : exacts à 10^{-6} DT, stables, calculés en quelques dizaines de microsecondes, et sans aucune offre invalide sur 12 912.
- **La lecture est de qualité** sur nos documents (100 % des champs, F1 0,997), et la classification en cascade est précise et peu coûteuse (96,2 %, 3,8 % d'appels externes).
- **La recherche hybride vaut son coût** : +9 points de rappel@5, surtout sur les valeurs exactes.

Ce que les résultats révèlent comme faiblesses

Faiblesse	Constat mesuré	Piste
Vitesse de l'OCR	44,5 s par page ; plus de 90 % du temps d'un dossier scanné	Un seul modèle de lecture selon la langue détectée, résolution adaptée, détection limitée, GPU, traitement en tâche de fond avec suivi
Erreur silencieuse de classification	Attestation arabe classée « justificatif de domicile » avec une confiance jugée suffisante	Consulter le modèle de langage quand les règles s'abstiennent ; relever le seuil des embeddings ; règles arabes enrichies
Disponibilité des fournisseurs de LLM	3 analyses sur 5 refusées par HTTP 429 (quota) pendant la mesure	Reprise automatique avec attente progressive, plan payant ou plus de fournisseurs, file d'attente des analyses
Score non reproductible corrigé	78 et 85 pour le même dossier avec le modèle de langage (écart-type 3,5) ; 0 avec la grille à points	Faire valider les poids et les seuils par la banque
Réponses de l'assistant	7 justes sur 9 ; une vraie erreur de montant	Re-classement des extraits, consigne de citer l'extrait, évaluation sur un échantillon plus large

Tableau 4.11 — Faiblesses constatées et pistes

Menaces sur la validité

- **Données fictives et propres.** Générées par nous, avec des titres proches de nos mots-clés : les taux de réussite sont optimistes.
- **Règles écrites en regardant les documents d'évaluation.** Risque de sur-ajustement : les 100 % de précision des règles ne se généralisent pas automatiquement.
- **Petits échantillons.** 26 documents, 22 questions, 12 réponses générées, 5 essais dont 2 aboutis : les pourcentages ont une grande marge d'incertitude.
- **Une seule machine, une seule session.** Les temps dépendent du matériel ; les modèles de langage externes varient dans le temps (quotas, charge).
- **Vérité terrain de l'OCR.** Le texte natif sert de référence : il suppose que le PDF natif est lui-même juste, ce qui est vrai pour des documents générés.

4.6 Synthèse à présenter en soutenance

Le tableau compare les mesures à des **objectifs proposés** (fixés pour ce rapport, à valider avec la banque : ils n'émanent pas d'une exigence contractuelle).

Étape	Indicateur	Objectif proposé	Mesuré	Verdict
-------	------------	------------------	--------	---------

OCR	Champs retrouvés (scan)	≥ 95 %	100 %	atteint
	Temps par page scannée	≤ 10 s	44,5 s	non atteint
Classification	Exactitude (cascade)	≥ 90 %	96,2 %	atteint (optimiste)
	Erreurs silencieuses	0	1	non atteint
	Part d'appels au LLM	≤ 20 %	3,8 %	atteint
	Latence des règles (médiane)	≤ 50 ms	2 ms	atteint
RAG	Rappel@5	≥ 90 %	95,5 %	atteint
	MRR	≥ 0,75	0,792	atteint
	Latence de recherche (médiane)	≤ 100 ms	20 ms	atteint
	Réponses justes	≥ 85 %	77,8 % (sur 9)	non atteint, échantillon faible
Décision	Écart du calcul à la référence	0 DT ($\leq 10^{-6}$)	0,000000 DT	atteint
	Offres invalides	0	0 / 12 912	atteint
	Latence du moteur de règles	≤ 1 ms	38 µs	atteint
	Analyses abouties (LLM externe)	≥ 95 %	2 / 5	non atteint (quota)
	Reproductibilité du score	écart-type ≤ 2	0 (grille, 2 000 répétitions) ; 3,5 avec l'ancien score du modèle	atteint (prototype)
Déploiement	à relever sur la VM (tableau 5.2)			

Tableau 4.12 — Mesures face aux objectifs proposés

Message à retenir. Les points **déterministes** (calculs, règles, recherche, classification par règles) sont fiables, rapides et testés. Les points qui dépendent d'un **modèle externe ou d'un calcul lourd** (OCR, modèle de langage) concentrent les faiblesses : vitesse, disponibilité, reproductibilité. L'architecture a justement été conçue pour que le modèle ne décide pas des chiffres : c'est ce qui explique que les valeurs restent exactes même quand le score varie.

Chapitre 5 — Déploiement

Ce chapitre décrit comment l'application est mise en ligne sur Microsoft Azure : l'infrastructure, la conteneurisation, l'automatisation, la configuration et les mesures à relever sur la machine.

5.1 Environnement cible

L'application tourne sur **une seule machine virtuelle Linux** d'un abonnement Azure étudiant. Ce choix est adapté à la charge attendue et au budget ; il a des conséquences (pas de haute disponibilité, une panne de la machine arrête le service) listées en 5.8.

Paramètre	Valeur par défaut (variables Terraform)
Taille de la machine	<code>Standard_B2as_v2</code> : 2 vCPU, 8 Go de RAM (famille « burstable »)
Région	<code>swedencentral</code> (une politique de l'abonnement étudiant limite les régions autorisées)
Disque système	64 Go
Système	Linux ; Docker installé au premier démarrage
Mémoire d'appoint	fichier d'échange de 4 Go : le service IA charge de gros modèles au démarrage
Nom de domaine	par défaut <code><adresse IP>.nip.io</code> si aucun domaine n'est fourni ; certificat HTTPS automatique

Tableau 5.1 — Environnement Azure

5.2 Infrastructure en tant que code (Terraform)

Terraform crée, de façon reproductible : le groupe de ressources (`credisense-rg`), le réseau virtuel et le sous-réseau, une adresse IP publique statique, un groupe de sécurité réseau (SSH sur le port 22, HTTP 80, HTTPS 443), l'interface réseau, la machine virtuelle et sa clé SSH. Les mots de passe de la base et le secret JWT sont **générés aléatoirement** (32 et 64 caractères). Un script `cloud-init` écrit les fichiers de configuration (`.env` , `ai.env` , `docker-compose.prod.yml` , `Caddyfile`) dans `/opt/credisense` , active le fichier d'échange, installe Docker et démarre PostgreSQL.

Contraintes rencontrées sur l'abonnement étudiant : région limitée par une politique, quota par famille de machines, erreurs de capacité (`SkuNotAvailable`). Les variables `location` et `vm_size` doivent être fixées ensemble, faute de quoi une nouvelle exécution tente de recréer la machine. Le script `recover-state.sh` et les commentaires de `terraform.tfvars.example` documentent ces cas.

5.3 Conteneurisation



Figure 5.1 — Les cinq services Docker Compose et leurs volumes

- **Images construites en plusieurs étapes** (compilation puis exécution) : Maven puis JRE 17 pour le backend, Node 22 pour le frontend, Python 3.11 pour le service IA. Dans l'image IA, PyTorch est installé en version processeur, les versions de PaddleOCR (3.4.0) et PaddlePaddle (3.3.1) sont alignées et les modèles sont **pré-téléchargés dans l'image** pour éviter un téléchargement au premier appel. La cause d'un échec de build antérieur était un décalage de versions Paddle.
- **Santé et ordre de démarrage** : PostgreSQL et le service IA exposent un contrôle de santé ; le backend attend que la base soit saine. Le service IA bénéficie d'un délai de grâce de 120 s (chargement des modèles).
- **Persistance** : la base (`pgdata`) et les fichiers des clients (`uploads` , partagé entre backend et service IA au même chemin) sont sur des volumes qui survivent aux redéploiements. Un défaut initial perdait les dépôts à chaque redéploiement ; il est corrigé.
- **Redémarrage automatique** (`unless-stopped`) de chaque service.

5.4 Intégration et déploiement continu

Le workflow GitHub Actions `deploy.yml` se déclenche à chaque envoi sur `main` ou `deploy/azure`. Il construit en parallèle les trois images (backend, frontend, ia) avec cache de construction, les publie sur le registre GHCR sous le numéro du commit et sous `latest`, copie les fichiers Compose sur la machine par SCP, puis s'y connecte par SSH pour télécharger les nouvelles images, mettre à jour `IMAGE_TAG` dans `.env` et relancer la pile (`docker compose up -d`), avant de purger les anciennes images. Sur la machine actuelle, les images ont aussi été construites localement (`IMAGE_TAG=latest` , option `--pull never`).

Étape	Action	Garantie
1	Construction des trois images en parallèle	Une image défaillante n'interrompt pas les autres (<code>fail-fast: false</code>)
2	Publication sous le SHA du commit	Retour arrière possible vers un commit précis
3	Mise à jour par SSH	Les secrets restent sur la machine : le workflow ne modifie que <code>IMAGE_TAG</code>

5.5 Configuration et secrets

Aucun secret n'est dans le code. Le fichier `.env` (mots de passe générés, JWT, clés, SMTP, domaine) et `ai.env` (clés des modèles de langage, taux d'intérêt) sont écrits sur la machine avec les droits `0600`. Les réglages de la notification sont des variables du backend : `AUTO_ENVOI_EMAIL`, `VALIDATION_REQUISE_POUR` et `DELAI_ENVOI_MINUTES` (voir l'annexe C). Le **taux d'intérêt annuel** `CREDIT_TAUX_ANNUEL` est obligatoire : sans lui, tout dossier reste « À COMPLÉTER ». La base se met à jour seule au démarrage du backend (`dd1-auto=update`), y compris la table du journal et les colonnes de notification.

5.6 Mise en ligne d'une version

1. Vérifier que `/opt/credisense/ai.env` contient le taux (`CREDIT_TAUX_ANNUEL=0.10` par exemple) et que le SMTP est renseigné pour l'envoi des réponses.
2. Envoyer le code sur la branche : GitHub Actions construit, publie et redémarre.
3. En cas de construction locale : `IMAGE_TAG=latest` puis `docker compose -f docker-compose.prod.yml up -d --force-recreate`.
4. Vérifier l'état (`docker compose ps`) et les journaux du backend (démarrage, création des tables).
5. Rafraîchir le navigateur (Ctrl + F5) pour charger la nouvelle version du frontend.

5.7 Mesures de déploiement

Un script en **lecture seule**, `deploy/mesurer_vm.sh`, relève sur la machine : ressources, taille des images, état et santé des conteneurs, consommation processeur et mémoire au repos, temps de démarrage lu dans les journaux, et latence HTTP de la page d'accueil, de l'API et du service IA (dix requêtes chacune, minimum, médiane, moyenne et maximum). Il ne redémarre rien, ne lit aucune donnée et n'affiche aucun secret.

Indicateur	Valeur mesurée sur la VM	Source
Taille de chaque image (backend, frontend, ia)	à relever	<code>docker images</code>
Mémoire utilisée au repos / disponible	à relever	<code>free -m</code> , <code>docker stats</code>
CPU et mémoire par conteneur	à relever	<code>docker stats --no-stream</code>
Temps de démarrage du backend	à relever	journal « Started ... in X seconds »
Latence page d'accueil (médiane)	à relever	10 requêtes <code>curl</code>
Latence API (médiane)	à relever	10 requêtes <code>curl</code>
Latence interne du service IA	à relever	10 appels à <code>/health</code>

Tableau 5.2 — Mesures de déploiement à relever sur la VM (commande à l'annexe E)

Pourquoi ces valeurs sont à relever. Elles dépendent de la machine réelle et ne peuvent pas être mesurées depuis un poste de développement. La commande de l'annexe E les produit en quelques secondes ; reportez-les dans ce tableau avant la soutenance.

5.8 Limites du déploiement

- **Une seule machine** : pas de haute disponibilité ; une panne ou une mise à jour interrompt brièvement le service.
- **Pas de sauvegarde automatisée** de la base ni du volume des fichiers : à mettre en place (instantané de disque ou export planifié).
- **Supervision** limitée aux contrôles de santé Docker et aux journaux ; pas d'alerte.
- **Accès SSH** ouvert à toutes les adresses par défaut (`allowed_ssh_source_ip = "*"`) : à restreindre.
- **Sécurité applicative** : voir 2.7.
- **OCR sur processeur** : la lenteur mesurée au chapitre 4 est directement liée à la taille de la machine ; une machine à processeur plus puissant ou un GPU réduirait fortement ce temps.

Conclusion générale et perspectives

Ce projet a produit une plateforme complète d'aide à la décision pour le crédit à la consommation : portail client, espace agent, administration, et un service d'intelligence artificielle couvrant la lecture des pièces, leur classification, l'extraction des données, un assistant documentaire et un moteur de décision. L'ensemble est testé automatiquement, déployé sur Azure de façon reproductible et mis à jour par une chaîne d'intégration continue.

Ce que le projet démontre

- **Une séparation nette entre lecture et décision.** Le modèle de langage lit et rédige ; mensualité, endettement, capacité, simulations, propositions d'ajustement et verdicts sont recalculés par des règles déterministes, mesurées au chapitre 4 comme exactes à l'arrondi près et sans aucune offre invalide.
- **Une chaîne qui préfère dire « il manque une information » plutôt que d'inventer.** Taux obligatoire, dette inconnue distincte de zéro, décision « À COMPLÉTER ».
- **Un humain dans la boucle et des comptes à rendre.** Validation avant envoi, confirmation des incohérences, journal d'audit en ajout seul avec la version des règles.
- **Des décisions de conception prises sur des mesures,** par exemple l'ajout d'un niveau de règles à la classification et la recherche hybride du chatbot.

Limites reconnues

- **Validation sur données fictives.** Les mesures prouvent le bon fonctionnement de la chaîne, pas sa précision sur de vrais dossiers : la qualité réelle de l'OCR, de la classification et de l'extraction ne sera connue qu'avec des documents bancaires réels.
- **Grille de score et seuils en prototype.** Le score est désormais calculé par le code (reproductible), mais ses poids et les seuils d'acceptation sont des valeurs par défaut à faire valider par la banque.
- **Vitesse de l'OCR sur processeur** (de l'ordre de 40 secondes par page) : acceptable en traitement différé, pas pour un usage interactif.
- **Seuils réglementaires non confrontés à une source officielle.**
- **Sécurité applicative et exploitation à finaliser** (routes ouvertes, secrets à renouveler, sauvegardes, supervision).

Perspectives

#	Axe	Contenu
1	Sécurité	Protéger toutes les routes, corriger la lecture de fichiers par chemin, retirer les données personnelles de l'historique git, renouveler les clés
2	Validation de la grille et des seuils	Faire valider par la banque les poids de la grille et chaque seuil (source, validateur, date) pour passer du statut « prototype » à « validé »
3	Jeu d'évaluation réel	Constituer, avec la banque, un corpus annoté (documents réels anonymisés) pour mesurer et recalibrer les seuils de la cascade

4	Performance de l'OCR	Résolution adaptée, détection limitée, un seul modèle de lecture selon la langue détectée, GPU
5	Conseiller de décision	Documents manquants demandés automatiquement au client ; scénarios pour les refus
6	Exploitation	Sauvegardes, supervision et alertes, conservation de la conversation de l'assistant en base, relances automatiques des dossiers « à compléter », politique de conservation des données (RGPD)

Annexes

Annexe A — Glossaire

Terme	Définition
Embedding	Représentation d'un texte par un vecteur de nombres ; deux textes de sens proche ont des vecteurs proches
Chunk	Morceau de texte indexé séparément pour la recherche
Indexation	Préparation des documents pour qu'on puisse les chercher vite (comme l'index d'un livre)
BM25	Classement par mots exacts, adapté aux montants, noms et numéros
RRF	Fusion de deux classements : $\text{score} = \sum \frac{1}{(60 + \text{rang})}$
Taux d'endettement (DTI)	Part du revenu net absorbée par les mensualités de crédit
Annuité constante	Mode de remboursement où la mensualité est identique chaque mois
Zone grise (classification)	Similarité entre 0,20 et 0,55 : les embeddings ne tranchent pas, le modèle de langage est consulté
Ajout seul (append-only)	Table où l'on ne fait qu'ajouter des lignes, jamais les modifier ni les supprimer

Annexe B — Points d'entrée de l'API

Service IA (FastAPI)

Route	Rôle
POST /ocr , /ocr/extract	Lecture d'un document
POST /ai/classify-hybrid	Cascade de classification et contrôle déclaré/détecté
POST /ai/extract-json	Extraction structurée selon le type
POST /ai/chat/index , /ai/chat	Indexation du dossier ; question à l'assistant
POST /ai/score/consommation	Moteur de décision
GET /health , /.../stats	Santé et statistiques d'usage

Backend (Spring Boot)

Route	Rôle
POST /api/public/demande , /api/public/upload	Demande et dépôt des pièces par le client

POST /api/fichiers/verifier-dossier/{id}	OCR, classification, extraction, cohérence du CIN
POST /api/fichiers/analyser-dossier/{id}	Analyse ; 409 sans confirmation d'une incohérence
GET /api/dossiers/{id}/resultat , /audit , /rapport-pdf	Résultat, journal, rapport PDF (agent ou client)
POST /api/dossiers/{id}/reponse/valider , /reponse/annuler	Contrôle humain avant l'envoi
POST /api/dossiers/{id}/send-result-email	Envoi à la demande, depuis le résultat enregistré
POST /api/chatbot/question	Assistant documentaire
GET /api/admin/statistiques , /export.xlsx	Tableau de bord et export (rôle ADMIN)

Annexe C — Variables de configuration

Variable	Défaut	Effet
CREDIT_TAUX_ANNUEL (service IA)	aucun	Taux annuel du crédit, en décimal (0.10 = 10 %). Obligatoire : sans lui, aucune mensualité ni décision
REGLES_PARAMETRES (service IA)	fichier livré	Chemin d'un autre fichier de paramètres (seuils et grille de score) ; un fichier invalide ramène aux valeurs internes
AUTO_ENVOI_EMAIL	true	Coupe tout l'envoi automatique si false (le bouton manuel reste)
VALIDATION_REQUISE_POUR	REFUS	Liste de décisions qui attendent une validation ; vide : aucune
DELAI_ENVOI_MINUTES	0	Délai annulable avant l'envoi des autres décisions
SMTP_USERNAME , SMTP_PASSWORD	vides	Messagerie ; sans elles, l'envoi échoue et l'agent le voit
DOMAIN , IMAGE_PREFIX , IMAGE_TAG	—	Domaine HTTPS, dépôt d'images et version déployée

Annexe D — Speech de présentation de l'architecture (environ 4 minutes)

Ouverture. Bonjour. Je vais vous présenter CrediSense AI, une application qui aide un conseiller bancaire à étudier une demande de crédit à la consommation. Le problème de départ est simple : un dossier arrive avec cinq ou six documents, souvent scannés, en arabe et en français, et le conseiller doit tout relire pour savoir si le client peut rembourser. Notre application fait cette lecture et ce calcul, puis explique son résultat.

Les trois blocs. L'architecture tient en trois blocs. Devant, une application Angular : un portail où le client dépose sa demande et ses pièces, et un espace où le conseiller travaille. Au milieu, un backend Spring Boot. C'est le chef d'orchestre : il gère les comptes, les dossiers, les fichiers, et il décide de l'ordre des étapes. Derrière, un service d'intelligence artificielle en Python avec FastAPI. Lui, il sait lire les documents, extraire les informations et faire l'analyse. Une base PostgreSQL garde tout, et un reverse proxy, Caddy, protège l'ensemble avec le HTTPS.

Pourquoi séparer le backend et le service IA. C'est un choix volontaire. Le monde de l'IA, avec PaddleOCR et les modèles, vit en Python. Le monde métier, avec la sécurité, les transactions et la base de données, est solide en Java. Chacun reste dans son langage et peut être déployé et mis à jour séparément. Ils se parlent par des appels HTTP simples, dans un réseau privé.

Le parcours d'un dossier. Le client dépose ses pièces. Le conseiller clique sur « Vérifier les documents ». À cet instant, chaque document passe par l'OCR : si le PDF contient déjà du texte, on le lit directement ; si c'est un scan ou une photo, PaddleOCR lit le texte avec deux modèles, un arabe et un français, puis on fusionne les deux lectures. Ensuite un modèle de langage transforme ce texte en données structurées : nom, CIN, salaire, type de contrat, avec des consignes adaptées au type de document. Et le système vérifie que le CIN lu sur chaque document est bien celui du client.

Le verrou. L'analyse ne démarre que si cette vérification est valide. Si le CIN ne correspond pas, le conseiller peut continuer, mais seulement en confirmant explicitement, et le serveur refuse de lui-même sans cette confirmation. Nous ne dépensons donc pas un appel d'IA sur un dossier qui n'est peut-être pas le bon.

Le contrôle des documents. Le client choisit lui-même où il dépose chaque pièce. Rien ne l'empêche de mettre un relevé bancaire à la place de la fiche de paie. Nous vérifions donc la nature réelle de chaque document, avec une classification en trois niveaux. D'abord des règles : le titre du document, « attestation de travail », « bulletin de paie », « relevé de compte », en français comme en arabe. C'est instantané, gratuit et reproductible. Si les règles ne sont pas sûres, on passe à une comparaison sémantique par embeddings, et seulement en dernier recours à un modèle de langage. Nous avons ajouté les règles après une mesure : les embeddings seuls classaient une attestation de travail comme une carte d'identité, parce qu'elle cite le numéro de carte. Si le contenu contredit l'emplacement choisi, l'analyse est bloquée jusqu'à ce que le conseiller confirme, et le type réel sert aussi à adapter l'extraction : on ne demande pas les mêmes champs à une carte d'identité et à un relevé.

Le chatbot. Le conseiller peut aussi poser des questions en langage naturel sur le dossier. Nous utilisons une architecture RAG : les documents sont découpés, transformés en vecteurs par un modèle d'embedding multilingue, et rangés dans un index. À chaque question, on retrouve les passages les plus proches, par le sens et par les mots exacts, puis un modèle de langage rédige la réponse en citant ses sources. Le découpage tient compte de la structure : un relevé bancaire est coupé par opération, avec ses en-têtes de colonnes, parce qu'un montant n'a de sens qu'avec sa colonne. Le chatbot a aussi une mémoire de conversation : le conseiller peut enchaîner « Et depuis quand ? » sans répéter le sujet, parce que la relance est complétée avec la question précédente avant la recherche.

Le cœur : un agent en deux couches. Pour l'analyse, nous avons fait un choix important. Le modèle de langage lit le dossier et propose une première évaluation. Mais il ne calcule pas les chiffres. Les mensualités, le taux d'endettement, la capacité d'emprunt et les simulations sont calculés par des règles, en Python, avec des formules connues. Ces règles contrôlent aussi les critères réglementaires : endettement sous 30 %, montant plafonné à cinq fois le salaire, durée, ancienneté. Si le modèle dit « éligible » alors qu'un critère bloquant échoue, les règles corrigent. Un chiffre financier doit être reproductible, et on peut l'expliquer.

L'honnêteté du calcul. Dernier point, et c'est celui dont je suis le plus fier : le système ne devine jamais. Si le taux d'intérêt n'est pas configuré, il ne calcule aucune mensualité. Si on ne sait pas si le client a déjà un crédit, il ne suppose pas que non : il détecte les échéances dans le relevé bancaire, et s'il n'y a pas de relevé, la dette reste inconnue. Dans ces cas, la décision n'est ni « éligible » ni « refus », c'est « à compléter », avec la liste précise de ce qu'il manque.

Déploiement. Tout tourne sur une machine virtuelle Azure, créée avec Terraform, donc reproductible. Les cinq services sont des conteneurs Docker orchestrés par Docker Compose. Les fichiers déposés et la base sont sur des volumes persistants, donc ils survivent aux mises à jour. Le code est construit et publié automatiquement par GitHub Actions.

Ce qui reste à faire, et je préfère le dire. Le score est calculé par une grille à points reproductible, mais ses poids et nos seuils sont ceux d'un prototype : ils doivent être validés par la banque. La classification est en place, mais nous ne connaissons pas encore sa précision réelle : nous l'avons mesurée sur nos propres documents de test, ce qui est optimiste. Il nous faut de vrais documents de la banque pour la valider. Côté sécurité, plusieurs routes doivent encore être protégées avant un usage réel. Et le taux d'intérêt doit être fourni par la banque.

Conclusion. En résumé : une architecture à trois couches claire, un pipeline verrouillé étape par étape, un calcul que l'on peut expliquer, et un système qui préfère dire « il me manque une information » plutôt que d'inventer une réponse. Merci, je suis prêt pour vos questions.

Questions probables et réponses courtes

Question	Réponse
Pourquoi un LLM et des règles ?	Le LLM comprend des documents variés. Les règles garantissent des calculs reproductibles et conformes. Chacun fait ce qu'il fait bien.
Est-ce vraiment un « agent » ?	Non au sens autonome. C'est un appel LLM suivi d'une couche de règles. Le pipeline orchestré par le backend joue le rôle de l'agent.

Que se passe-t-il si le LLM est saturé ?	Bascule automatique Groq / Mistral. Si tous échouent, la décision est « indéterminée » avec un message clair, sans bloquer le dossier.
Que se passe-t-il si un document est illisible ?	Le statut OCR « FAILED » et sa raison s'affichent sur le document. Aucune extraction vide n'est enregistrée comme valide.
Comment gérez-vous l'arabe ?	Deux lectures de la même zone, par un modèle arabe et un modèle latin, fusionnées zone par zone.
Les données personnelles sont-elles protégées ?	Le chatbot masque RIB, IBAN et CIN, et les journaux n'affichent plus le texte OCR. Les routes API doivent encore être protégées : c'est notre priorité de sécurité.
Quelle est la précision ?	Mesurée le 8 octobre sur nos documents fictifs. OCR : 18 champs sur 18 retrouvés, F1 des mots 0,997. Classification : 96,2 % (25 sur 26) avec la cascade, contre 61,5 % avec les embeddings seuls. Recherche du chatbot : rappel@5 de 95,5 %. Calculs : écart nul à une référence indépendante, aucune offre invalide sur 12 912. Ces chiffres sont optimistes (documents écrits par nous) : la vraie précision se mesurera avec des documents réels de la banque. Score : il variait avec le modèle de langage (78 et 85 pour le même dossier) ; il est maintenant calculé par une grille à points et ne varie plus (écart-type 0), mais les poids restent un choix de prototype à valider avec la banque.
Un refus peut-il partir sans qu'un humain l'ait vu ?	Non par défaut : un refus attend « Valider et envoyer » de l'agent, qui peut aussi décider de ne pas l'envoyer. La validation porte sur la décision lue : si une nouvelle analyse l'a changée, le serveur refuse. Les autres décisions peuvent partir seules ou après un délai annulable, c'est réglable.
Comment prouvez-vous qui a décidé quoi ?	Un journal d'audit en ajout seul : chaque analyse (décision, score, version des règles, fournisseur d'IA) et chaque étape de la réponse (mise en attente, validation, annulation, envoi, échec) est consignée avec l'agent et l'heure. Aucune route ne permet de le modifier.
Que montre le tableau de bord ?	Taux d'acceptation (éligibles ÷ décisions définitives, avec une version « avec conditions »), répartition des décisions, délais de traitement et d'envoi, motifs de refus, évolution dans le temps et état des envois, avec un export Excel. Les définitions sont affichées à l'écran.
Comment fonctionne votre RAG ?	Indexation : découpage structurel et sémantique, embeddings MiniLM multilingue, index FAISS en mémoire plus BM25. Question : recherche hybride (sens plus mots exacts) fusionnée par RRF, au moins 2 passages par document, puis génération par le LLM avec sources citées et identifiants masqués.
Comment gérez-vous la mémoire du chatbot ?	Le navigateur renvoie les 6 derniers messages avec chaque question. Le modèle les reçoit comme historique de conversation, et une question de suivi est complétée avec la question précédente pour la recherche, par une règle simple qui n'ajoute aucun appel LLM. L'historique n'est pas cru : rôles filtrés, identifiants masqués, messages tronqués.

Pourquoi une recherche hybride ?	Les embeddings comprennent le sens mais pèsent mal les valeurs exactes. Sur « 250,000 », la similarité sémantique n'est que de 0,27, alors que BM25 retrouve immédiatement l'opération. Les deux sont complémentaires.
Que se passe-t-il si le client dépose le mauvais document ?	La classification le détecte, le pipeline affiche « déposé comme fiche de paie, le contenu ressemble à relevé bancaire », et « Analyser » reste bloqué jusqu'à ce que le conseiller confirme. Si la classification n'est pas sûre, elle ne bloque rien.
Quelle est la vitesse du système ?	Les étapes déterministes sont instantanées (règles du moteur : 38 microsecondes ; recherche : 20 ms). L'OCR d'un scan prend en moyenne 44,5 secondes par page sur processeur : c'est plus de 90 % du temps d'un dossier scanné, donc la première piste d'optimisation.
Quelle est la faiblesse mesurée la plus grave ?	Une erreur silencieuse de classification : une attestation d'emploi arabe classée « justificatif de domicile » avec une confiance jugée suffisante. Et la disponibilité des fournisseurs de modèles de langage : 3 analyses sur 5 ont reçu un refus de quota pendant la mesure. Les deux ont des pistes de correction.
Pourquoi appeler cela un « moteur de décision hybride » ?	Hybride : un modèle de langage lit et rédige, des règles calculent et tranchent. Moteur : il applique des règles versionnées et testables, sans autonomie. Le nom « agent » laissait croire à une décision autonome.

Annexe E — Commandes de reproduction des mesures

Mesures d'une étape (poste de développement, depuis le dossier `Python/`, environnement virtuel activé)

```
python benchmarks/mesurer_kpi.py ocr
python benchmarks/mesurer_kpi.py classification # ajouter --sans-llm pour éviter tout
appel réseau
python benchmarks/mesurer_kpi.py rag
python benchmarks/mesurer_kpi.py decision --runs 5
python benchmarks/mesurer_kpi.py all # tout enchaîner (long : 1'OCR des
scans prend plusieurs minutes)
```

Chaque commande écrit `Python/benchmarks/resultats/kpi_<étape>.json`.

Tests automatisés

```
cd Python && python -m unittest discover -s tests
cd Backend/backend && ./mvnw -q -o test # JDK 17 à 20 (JDK 23 est incompatible avec
Lombok)
cd frontend && npx ng test --watch=false && npx ng build
```

Mesures de déploiement (sur la machine virtuelle, depuis Azure Cloud Shell, lecture seule)

Téléverser `deploy/mesurer_vm.sh` dans Cloud Shell (bouton « Charger / Télécharger »), puis :

```
az vm run-command invoke -g CREDISENSE-RG -n credisense-vm \  
--command-id RunShellScript --scripts @mesurer_vm.sh
```

La sortie est un JSON dont le champ `message` contient les relevés ; reporter les valeurs dans le tableau 5.2.